TODO

- [ ] proof read, stavefeil, flow, struktur, ordlegging akademisk tone. formler, gjenta til fornøyd
- [ ] figurer
- [ ] referanser
- [ ] metode og resultater handcrafted/kopierte vekter
- [ ] metode og resultater neuromorfisk læring
- [ ] discusion
- [ ] conclusion
- [ ] abstract
- [ ] siste proof read

UNIVERSITY
OF OSLO

Master's Thesis

NEUROMORPHIC COMPUTING

With Spiking Neural Networks

Brage Wiseth

Departement of Informatics
Faculty of Mathematics and Natural Sciences

Philip Haflinger - Supervisor
Yngve Hafting - Co-Supervisor

2026-03-16

ABSTRACT

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae.

ACKNOWLEDGEMENTS & DECLARATIONS

I would like to thank my supervisor and the very kind and helpful comunity at ROBIN and NANO labratories at the departemnt of informatics. I would like to that professor Farad at SDU

DECLARATION OF THE USE OF GENERATIVE ARTIFICIAL INTELLIGENCE

In this scientific work, generative artificial intelligence (AI) has been used. All data and personal information have been processed in accordance with the University of Oslo's regulations, and I, as the author of the document, take full responsibility for its content, claims, and references. An overview of the use of generative AI is provided below.

The service GPT UiO, developed by UiO IT department, has been used to improve the content of the report/assignment. A first version of the work was pasted in its entirety, and the model was given the prompt [rewrite this text to make the language more lively.] The text was then iterated a few times through the model where new prompts were used to get the correct structure of the text. The final result was cut out, fact-checked, and partly rewritten by the author(s).

CONTENTS

Wordcount: 22806

GLOSSARY

***AI* – Artificial Intenligence** 5, 6, 6, 7, 7, 56

***ANN* – Artificial Neural Network** 10, 29, 63

action potential: Voltage potential in the neuron cell membrane that propagates trough the neurons axon and to its synapses 15

***CNN* – Convolutional Neural Network** 11, 29, 71, 71, 71

***CPU* – Central Processing Unit** 6

***DAG* – Directional Acyclic Graph** 29, 34

***DL* – Deep Learning** 43

***GLIF* – Generalized Leaky Integrate and Fire** 21, 21, 21, 22, 22

***GPT* – Generative Pre-trained Transformer** 11

***GPU* – Graphics Processing Unit** 5, 6, 11

***LIF* – Leaky Integrate and Fire** 19, 20, 20, 21, 21

***LLM* – Large Language Model** 11

***LSTM* – Long Short-Term Memory** 11

***LTD* – Long-Term Depression** 40, 41

***LTP* – Long-Term Potentiation** 40, 41, 41

***MLP* – Multi Layer Perceptron** 9, 10, 10, 11

***PSC* – Post Synaptic Current** 28

***RNN* – Recurrent Neural Network** 11

***SNN* – Spiking Neural Network** 28, 55, 63, 63, 71, 71, 71

***STDP* – Spike-Timing-Dependent Plasticity** 40, 40, 40, 41, 41, 41, 42, 58, 58, 59, 62, 63, 63, 73

***TTFS* – Time-To-First-Spike** 25, 25, 25, 25, 27, 66, 67, 70, 80, 80, 80, 81

***VLSI* – Very Large Scale Integration** 13

***WTA* – Winner-Take-All** 25, 31, 31

1 • INTRODUCTION

The development of intelligent machines is a significant objective in modern science and engineering. While the concept has historical roots in philosophy and early automata, the field has transitioned from speculative theory to practical application. Currently, artificial intelligence is central to technological and economic development. Understanding the mechanisms of intelligence and reproducing them in synthetic systems offers the potential for improved analysis of biological minds and the creation of tools for applications ranging from personalized medicine to automated scientific discovery.

In recent years, great strides has been made towards this goal. Deep Learning, which utilizes multilayered neural networks, has exceeded previous performance benchmarks. These systems have demonstrated high proficiency in tasks previously limited to human capability. For example, AlphaFold has addressed complex problems in protein folding [1], reinforcement learning agents have mastered the complexity of games such as Go [1], and Large Language Models have shown capabilities in text generation that approach human fluency. Consequently, Artificial Intenligence (AI) is increasingly viewed as a general-purpose technology that may influence societal infrastructure.

However, despite these advances, there are significant limitations to the current approach. The success of modern deep learning relies heavily on scaling, which involves increasing data volume and computational power. This strategy is approaching physical and economic boundaries. Training state-of-the-art models consumes substantial energy and results in a large carbon footprint [1]. Although specialized hardware allows for more efficient computations, the underlying architecture and algorithms imposes an intrinsic limit on scalability independent of the underlying hardware. Furthermore, the requirement for massive datasets presents challenges in sourcing and curation. Additionally, evidence suggests that this scaling approach yields diminishing returns. Models often function as statistical correlation engines; they lack common-sense reasoning, struggle with out-of-distribution generalization, and are prone to brittle failure modes [1].

These limitations are evident when comparing artificial systems to biological intelligence. The human brain demonstrates that high-level intelligence is possible without massive energy consumption or dataset sizes. The brain operates on approximately 20 watts [1]. With this limited energy budget, it manages biological functions, processes real-time multi-sensory data, and performs abstract reasoning. In contrast, deep learning models require Graphics Processing Unit (GPU) clusters with significantly higher power requirements to match a fraction of these capabilities. There is also a discrepancy in learning efficiency. Deep learning models are sample-inefficient, often requiring vast numbers of examples to learn a representation. Biological systems, however, are capable of “one-shot” or “few-shot” learning

and can acquire new information without catastrophic forgetting. This suggests the inefficiency of current AI is a paradigmatic issue rather than just an engineering problem.

The proposed direction for addressing these issues involves biological inspiration in both hardware and algorithm design, specifically Neuromorphic Computing. This field attempts to engineer computer architectures that mimic the biological structure of the nervous system. Unlike traditional AI, which runs as software on general-purpose hardware, neuromorphic engineering aims to align the algorithm with the physical substrate. It moves away from clock-driven processing toward asynchronous, event-driven systems. In this paradigm, information is encoded as sparse, discrete events or "spikes." Similar to biological neurons, a neuromorphic processor consumes minimal energy when inactive, processing information only when triggered. This approach is being pursued by both industrial groups, such as Intel's Loihi [1], and academic projects like SpiNNaker [1]. These systems represent a shift from calculation-based machines to those capable of real-time adaptation.

Although neuromorphic systems achieve optimal performance on co-designed platforms—where the algorithm is embedded directly into the hardware—there is significant value in executing neuromorphic algorithms on traditional von Neumann architectures. In this thesis, we explore biologically inspired algorithms deployed on traditional Central Processing Unit (CPU) and GPU hardware. We examine how event-driven, biologically plausible computation can address limitations in scalability, data efficiency, and energy consumption, even when simulated on standard processors. We present approaches for efficient information coding and learning algorithms inspired by neural mechanisms. Concretely, this thesis aims to:

- **Investigate Sparse Information Flow**: Explore how information can be encoded and processed using sparse, asynchronous events (spikes) within a neural network.
- **Develop Biologically Inspired Learning**: Explore and evaluate learning algorithms that are suitable for such networks, adhering to the constraints of locality and efficiency.

The succeeding chapter lays the historical and theoretical foundation, covering early neuroscience and the development of artificial neural networks based on simple models of the brain. Following this, we review relevant modern neuroscience literature, extracting key concepts that will inform the methodology. We also provide consise overview on machine learning concepts and frameworks. Finally, we detail the implementation of these principles in a neuromorphic context and evaluate their performance against standard benchmarks.

2 • BACKGROUND

This section outlines the historical and theoretical evolution of AI, reviewing key concepts in modern neuroscience that motivate the methodology used in this thesis.

We begin at a shared origin point, a time when AI research and neuroscience were intertwined. We then trace the diverging path that led to modern Deep Learning, examining why it has drifted from biological plausibility. Subsequently, we explore the "neuromorphic path". In Section 2.2, we detail the specific physical principles and neuroscientific insights upon which the neuromorphic methods in this thesis is built. In Section 2.3, we contrast the architectural mechanics of deep learning and neuromorphic systems, specifically addressing why the former is computationally powerful yet energetically inefficient. We conclude with a review of existing frameworks, identifying their strengths and weaknesses to contextualize the contributions of this work.

2.1 • HISTORY & DEVELOPMENTS

Historically, the understanding of neural tissue was dominated by the reticular theory, which claimed that the brain consisted of a continuous, fused network of nerve fibers. This paradigm was fundamentally challenged by the work of Santiago Ramón y Cajal. Through the application of novel staining techniques, Cajal established the neuron doctrine, demonstrating that the nervous system is composed of discrete, individual cells [1]. Building on these findings, Heinrich Wilhelm Gottfried Von Waldeyer-Hartz proposed the "Neuron Doctrine" and coined the term "neurons" to describe these dicrete cells. Subsequent analysis using electron microscopy has provided irrefutable validation of this discrete cellular structure.

The conceptualization of the brain as a collection of discrete units facilitated the development of mathematical models describing neural function. In 1943, Warren McCulloch and Walter Pitts published A Logical Calculus of the Ideas Immanent in Nervous Activity, introducing the first formal model of the neuron.

The McCulloch-Pitts (M-P) neuron abstracted biological complexity into a binary decision device governed by the following logic:

- **Inputs**: The neuron receives multiple binary inputs, weighted as either excitatory or inhibitory.
- **Summation**: The unit calculates the linear sum of these weighted inputs.
- **Thresholding**: If the aggregate sum exceeds a fixed threshold, the neuron outputs a 1 (firing); otherwise, it outputs a 0 (silence).

McCulloch and Pitts demonstrated that networks of these units could theoretically compute any logical operation (AND, OR, NOT) [1]. This abstraction established the foundational link between biological processes and digital logic, suggesting that neural function could be replicated in electronic hardware. Consequently, the M-P neuron serves as the common ancestor for both computational neuroscience and artificial intelligence.

However, despite its theoretical utility, the original M-P model presented significant functional limitations. The connectivity was static, requiring circuits to be manually designed rather than learned. Furthermore, the restriction to binary weights precluded the modeling of graded signal intensity, preventing the system from capturing the nuance of real-world sensory input.

In 1949, Donald Hebb addressed the critical issue of plasticity in his work The Organization of Behavior. He proposed a theoretical mechanism for synaptic modification, now known as Hebbian learning, which provided a biological basis for how neural networks could adapt over time. Hebb postulated:

> "Let us assume that the persistence or repetition of a reverberatory activity (or "trace") tends to induce lasting cellular changes that add to its stability. ... When an axon of cell A is near enough to excite a cell B and repeatedly or persistently takes part in firing it, some growth process or metabolic change takes place in one or both cells such that A's efficiency, as one of the cells firing B, is increased" [1].

This principle is colloquially summarized as "neurons that fire together, wire together" [1]. Crucially, this describes a local and decentralized learning rule; a synapse does not require a global error signal or external supervision to adjust. It requires only the correlation between the pre-synaptic input and the post-synaptic output. The convergence of the M-P architectural model and the Hebbian plasticity framework established the prerequisite conditions for the development of modern neural networks.

2.1.1 • THE PERCEPTRON

In 1957, Frank Rosenblatt advanced these theoretical concepts by engineering the Perceptron. The "Mark I Perceptron" was a hardware implementation of the neural model, distinguished by a crucial innovation: a weight-adjustment mechanism based on Hebbian principles. Rosenblatt introduced the perceptron learning rule, an iterative algorithm capable of minimizing error automatically. The system processed an input pattern (e.g., a pixelated character) and produced a binary classification. When the output deviated from the target, the algorithm adjusted the weights proportional to the error: strengthening connections that should have contributed to a correct firing and weakening those that led to false positives.

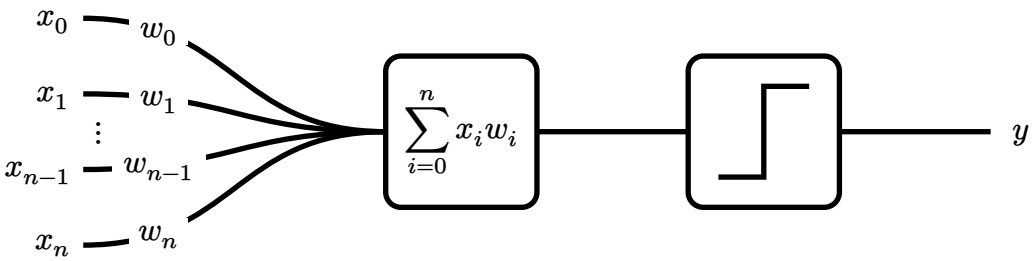


Figure 1: The perceptron model. Inputs x_i are multiplied by weights w_i and summed. If the linear combination $\sum x_i w_i$ exceeds the bias b, the neuron activates.

Consequently, the Perceptron was capable of converging on a solution for any problem where the data was linearly separable. This success generated significant enthusiasm, with contemporary reports suggesting that such machines would soon mimic human consciousness [1].

These expectations were abruptly tempered by theoretical limitations. In 1969, Marvin Minsky and Seymour Papert published Perceptrons, a rigorous mathematical analysis of the architecture. They demonstrated that a single-layer perceptron is fundamentally a linear classifier. While capable of learning operations like AND or OR, it is mathematically incapable of solving the XOR (Exclusive OR) problem. In the XOR case, the classes cannot be separated by a single hyperplane. This proof highlighted a severe boundary on the utility of single-layer networks for complex, non-linear tasks.

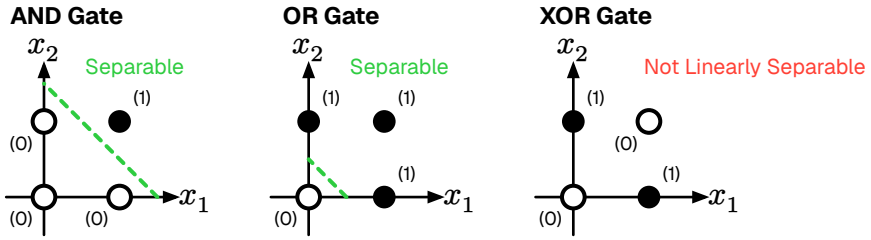


Figure 2: The XOR problem. Unlike AND/OR, the data points for XOR cannot be separated by a single linear boundary.

The publication of Perceptrons coincided with a significant reduction in neural network research funding, a period retrospectively termed the “First AI Winter”. It is worth noting that Minsky and Papert acknowledged that a Multi Layer Perceptron (MLP), a network stacking multiple layers of neurons, could theoretically solve the XOR problem by creating complex, non-linear decision boundaries.

However, a critical algorithmic gap remained: the “credit assignment problem”. While researchers knew that hidden layers could represent complex features, there was no known method to propagate error signals back through the layers to adjust the weights of hidden neurons effectively. Rosenblatt’s rule was mathematically valid only for the output layer. The field remained stagnant until a method for training multi-layer networks could be formalized.

2.1.2 • DEEP LEARNING

The critique presented by Minsky and Papert precipitated a contraction in funding; despite this, theoretical inquiry persisted. It was widely hypothesized that the limitations of the single perceptron could be overcome by a MLP. By organizing neurons into hierarchical layers, the network could theoretically perform successive non-linear transformations on the input space, enabling the formation of complex decision boundaries. The primary impediment was not the architecture itself, but the absence of a viable learning algorithm.

In a single-layer perceptron, error attribution is immediate: if the output deviates from the target, the error is directly derived from the weights of the output layer. However, in a multi-layer architecture, quantifying the contribution of a specific neuron within the "hidden" layers to the final output error presents a significant challenge. This is formally known as the Credit Assignment Problem [1], and it remained the central theoretical obstacle for over a decade.

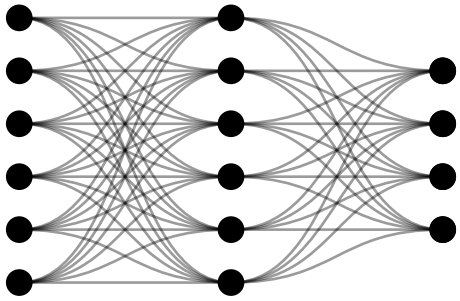


Figure 3: A MLP. By inserting "hidden layers" between input and output, the network can approximate non-linear functions such as XOR. The historical challenge lay in deriving a method to train these intermediate layers.

The solution to this theoretical impasse was popularized in 1986 by Rumelhart, Hinton, and Williams in their seminal paper *Learning representations by back-propagating errors* [1]. They demonstrated that the Chain Rule of calculus could be applied recursively to propagate the error signal from the output layer backwards through the hidden layers. This algorithm, known as Backpropagation, allowed the network to calculate the gradient of the loss function with respect to every weight in the system. Effectively, it provided a mathematical method to tell each hidden neuron exactly how much it contributed to the total error, finally solving the credit assignment problem.

Unlike Hebbian plasticity, which is local and biological, Backpropagation relies on global error signals and precise backward data flow—mechanisms effectively absent in organic tissue. Consequently, the field of Artificial Neural Network (ANN) effectively decoupled from neuroscience. It transitioned into a branch of engineering and applied mathematics, prioritizing statistical optimization over biological realism. Paradoxically, it was this abandonment of biological fidelity that enabled the rapid scaling and performance breakthroughs that followed.

ACHIEVEMENTS

With the training mechanism solved, the field exploded. The combination of Backpropagation, massive datasets, and GPU hardware led to a "Cambrian Explosion" of neural architectures, each solving domains previously thought impossible for computers.

The revolution began in earnest with computer vision. Convolutional Neural Networks (CNNs), such as AlexNet (2012) [1] and later ResNet [1], introduced the idea of learning hierarchical features—detecting edges, then shapes, then objects—much like the human visual cortex. This allowed machines to classify images with superhuman accuracy.

Soon after, the focus shifted to sequence data. Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) architectures gave machines a short-term memory, enabling breakthroughs in speech recognition and machine translation. However, the true paradigm shift occurred with the introduction of the Transformer architecture in 2017. By utilizing an "attention mechanism" to parallelize the processing of language, Transformers allowed for the training of massive Large Language Models (LLMs) like the Generative Pre-trained Transformer (GPT).

These techniques have even transcended media generation. Deep Learning has solved fundamental scientific problems; notably, DeepMind's AlphaFold utilized these architectures to predict the 3D structure of proteins from their amino acid sequences, a 50-year-old grand challenge in biology [1].

SHORTCOMINGS

Deep learning has achieved substantial success across various domains. However, this performance relies heavily on computational scaling. The underlying algorithms, while originally inspired by biological principles, have diverged significantly to prioritize mathematical optimization on standard hardware. By simulating neural networks on architectures not designed for them—using algorithms developed to address the specific constraints of the MLP rather than the broader goal of efficient intelligence—this approach faces fundamental barriers that cannot be overcome merely by increasing hardware resources.

A primary limitation stems from the underlying computer architecture. Modern systems rely on the Von Neumann architecture, which physically separates the processing unit from the memory. Deep neural networks, which are defined by large matrices of synaptic weights, require constant data transfer between these components. For every token generated or inference step performed, billions of weight parameters must be fetched from memory, processed, and written back. This creates a memory bottleneck where system performance is limited not by processing speed, but by the available bandwidth to memory [1].

Concurrently, the computational cost of training state-of-the-art models is growing exponentially. Deep learning relies on dense matrix multiplications, where the number of operations scales quadratically with the network size. As models grow to encompass trillions of parameters to achieve marginal gains in performance, the hardware requirements become economically and physically unsustainable. This reliance on brute-force scaling yields diminishing returns, suggesting that the current architectural paradigm is approaching an efficiency plateau.

This architectural mismatch manifests acutely in energy consumption. In modern CMOS technology, the energy cost of moving data significantly exceeds the cost of processing it. Retrieving a single byte of data from off-chip DRAM consumes approximately three orders of magnitude more energy than performing a floating-point operation on that data [1]. As highlighted in the introduction, this discrepancy has led to the proliferation of "Megawatt Models." In stark contrast, the human brain operates on approximately 20 watts. By co-locating memory and computation, biological systems manage complex multimodal processing with an energy budget comparable to a dim lightbulb.

Furthermore, backpropagation-based learning exhibits low sample efficiency compared to biological systems. Deep learning models often require millions of examples to establish robust representations, whereas biological agents demonstrate "one-shot" or "few-shot" learning capabilities. Additionally, the resulting models function as "black boxes." While empirically effective, their distributed internal representations are often opaque, making it difficult to trace the causal logic behind specific decisions or failures.

These limitations can be traced to the deviation from biological constraints. To solve the training problem, mainstream AI adopted mechanisms that are biologically implausible. Backpropagation relies on a global error signal and requires the backward pass to use the exact same synaptic weights as the forward pass (the "weight transport problem"). In biological tissue, synapses are unidirectional, and there is no known mechanism for a neuron to access the weight of a downstream synapse to calculate a gradient.

While a detailed technical analysis of these inefficiencies is presented in Section 2.3, the broader implication is clear: we have achieved artificial intelligence at the cost of efficiency and explainability. This realization has renewed interest in alternative architectures that align more closely with biological principles. Although the majority of research has focused on deep learning, a parallel subset of the field has continued to investigate systems that mimic the physical operation of the nervous system.

2.1.3 • BIRTH OF NEUROMORPHIC

While the artificial intelligence community debated symbolic logic versus connectionism during the "AI Winter," significant developments were occurring in hardware physics. In the late 1980s at Caltech, physicist Carver Mead—a pioneer

of Very Large Scale Integration (VLSI) design—began to question the trajectory of digital computing.

Mead observed that while digital computers were becoming exponentially faster, they were also becoming less efficient in terms of energy per operation. He noted that using transistors as rigid, high-power switches to perform boolean logic was energetically wasteful compared to the biological systems they aimed to emulate.

In 1990, Mead published his seminal paper, *Neuromorphic Electronic Systems* [1], coining the term "neuromorphic" to describe hardware that mimics the biological structure of the nervous system. His thesis proposed that rather than simulating neural equations via software on digital computers, engineers should construct physical hardware that exploits the same physical laws as the biological nervous system.

The foundational insight of the field was the physical analogy between silicon physics and ion-channel physics. In standard digital electronics, transistors are operated in "strong inversion," driven by high voltages to act as binary switches. Mead realized that a single transistor, operating in its "subthreshold" region, follows the same exponential Boltzmann statistics that govern the flow of ions through biological channels.

This realization implied that a single transistor could physically compute the non-linear functions used by biological neurons, but with significantly higher speed and lower power consumption. Consequently, synaptic functions could be implemented by single transistors rather than complex arrangements of logic gates.

To demonstrate this concept, Mead and his doctoral student Misha Mahowald developed the *Silicon Retina* in 1991 [1]. Unlike a standard camera, which captures full frames at fixed intervals (generating redundant data), the Silicon Retina operated asynchronously. It utilized analog circuits to compute spatial and temporal derivatives directly on-chip, outputting discrete "events" only when local light intensity changed.

This event-driven approach solved the redundancy problem inherent in frame-based sampling. If the scene remained static, the system transmitted no data and consumed negligible energy. This demonstrated that by aligning the hardware physics with the computational task, sensory information could be processed with a fraction of the power required by conventional digital systems.

Since the inception of neuromorphic computing, neuroscience has also advanced significantly. While Mead's early work was based on the physical intuition of the transistor, modern neuromorphic engineering now incorporates a richer understanding of neuronal dynamics, synaptic plasticity, and network architecture. To advance the field, we must combine these foundational hardware insights with the principles of modern mechanistic neuroscience.

2.2 • BIOLOGICAL PRINCIPLES

The biological brain remains the gold standard for energy-efficient, robust, and adaptive computation. Since the establishment of the Neuron Doctrine, modern neuroscience has uncovered the specific physical mechanisms that underpin this efficiency. To engineer systems that truly rival biological performance, we must transcend the "spherical cow" abstractions of early cybernetics. We cannot simply mimic the brain's output; we must emulate its internal dynamics. This requires viewing the neuron not as a static summing unit, but as it functions in reality: a complex, time-dependent, and event-driven processor.

This section provides a mechanistic overview of the nervous system, translating biological observations into the computational primitives required for neuromorphic engineering. It explores the structural hierarchy of the neuron, the physics of the action potential, and the mathematical models used to capture these dynamics in silicon.

2.2.1 • NEURON STRUCTURE & FUNCTION

In Section 2.1 we established the neuron as the fundamental computational unit of the brain. While it shares standard cellular machinery like mitochondria and a nucleus with other cells, it is morphologically specialized for information transmission. A neuron consists of three functional zones:

- **The Input (Dendrites)**: A branching tree structure that collects signals from thousands of upstream neurons. This is where inputs are integrated.
- **The Integration Zone (Soma)**: The cell body where electrical potentials from the dendrites summate.
- **The Output (Axon)**: A long, cable-like structure that transmits the neuron's own signal to downstream targets.

The neuron exhibits a distinct morphological polarization that dictates the direction of information flow. The process begins at the "dendritic arbor", a complex branching structure that maximizes the surface area for synaptic connectivity. These dendrites serve as the primary receptor sites, where neurotransmitters binding to post-synaptic terminals induce local conductance changes. These signals propagate passively toward the soma (cell body), the neuron's central processing unit. The soma acts as an integrator, spatially and temporally summing the incoming synaptic currents. Finally, the processed signal is transmitted via the axon, a singular, elongated projection. In many vertebrate neurons, the axon is insulated by a myelin sheath, which facilitates saltatory conduction—a mechanism that allows high-speed signal propagation over long distances with minimal signal degradation.

Figure 4: The morphological structure of a biological neuron, illustrating the directional flow of information from dendritic input to axonal output.

Functionally, the neuron operates as an electrochemical system enclosed by a cell membrane, known as the "lipid bilayer". This membrane is a thin, fatty structure that is impermeable to ions, acting as an electrical insulator. However, the fluids inside and outside the cell are conductive electrolytes. Consequently, the interaction between the insulating membrane and the conductive fluids creates a biological capacitor, capable of storing charge.

By actively pumping sodium (Na^+) out and potassium (K^+) in via the Na^+-K^+ ATPase pump, the cell maintains an electrochemical gradient across this capacitor, resulting in a stable "resting potential" of approximately -70 mV.

Computation occurs through the modulation of this voltage by competing synaptic inputs. excitatory inputs cause ion channels to open, allowing positive ions to influx; this reduces the negative charge (depolarization) and pushes the potential toward the firing threshold. Conversely, inhibitory inputs activate channels for negative ions (like Chloride, Cl^-), driving the potential away from the threshold (hyperpolarization). The soma integrates these opposing push and pull signals. If the aggregate membrane potential surpasses a critical threshold (approximately -55 mV), the system undergoes a bifurcating phase transition. Voltage-gated sodium channels cascade open, triggering an action potential—a rapid, non-linear depolarization spike that propagates down the axon. This mechanism is governed by the

"all-or-nothing" principle: the output is discrete and binary, effectively filtering out sub-threshold noise.

Immediately following a spike, the neuron enters a "refractory period" during which ion gradients are restored, imposing a hard limit on the maximum firing frequency and ensuring the temporal separation of events.

It is important to acknowledge that the biological brain exhibits significant cellular diversity beyond this idealized model. The nervous system contains non-neuronal cells known as "glia", which provide structural support and manage energy delivery, though they are generally not considered direct participants in fast information transmission. Additionally, while the vast majority of cortical neurons communicate via uniform action potentials (spikes), certain sensory neurons utilize "graded potentials", where the signal amplitude varies continuously. However, as spiking neurons represent the dominant computational paradigm for information processing in the cortex, this thesis focuses exclusively on the spiking model as the basis for neuromorphic emulation.

2.2.2 • ACTION POTENTIAL & SPIKE TRAINS

As established in the previous section, the action potential is an "all-or-nothing" event. It serves as the fundamental mechanism by which neurons transmit information. Crucially, the waveform of this event is stereotypical: for a given neuron, every spike exhibits a nearly identical amplitude and duration (typically 1–2 ms), independent of the input intensity that triggered it.

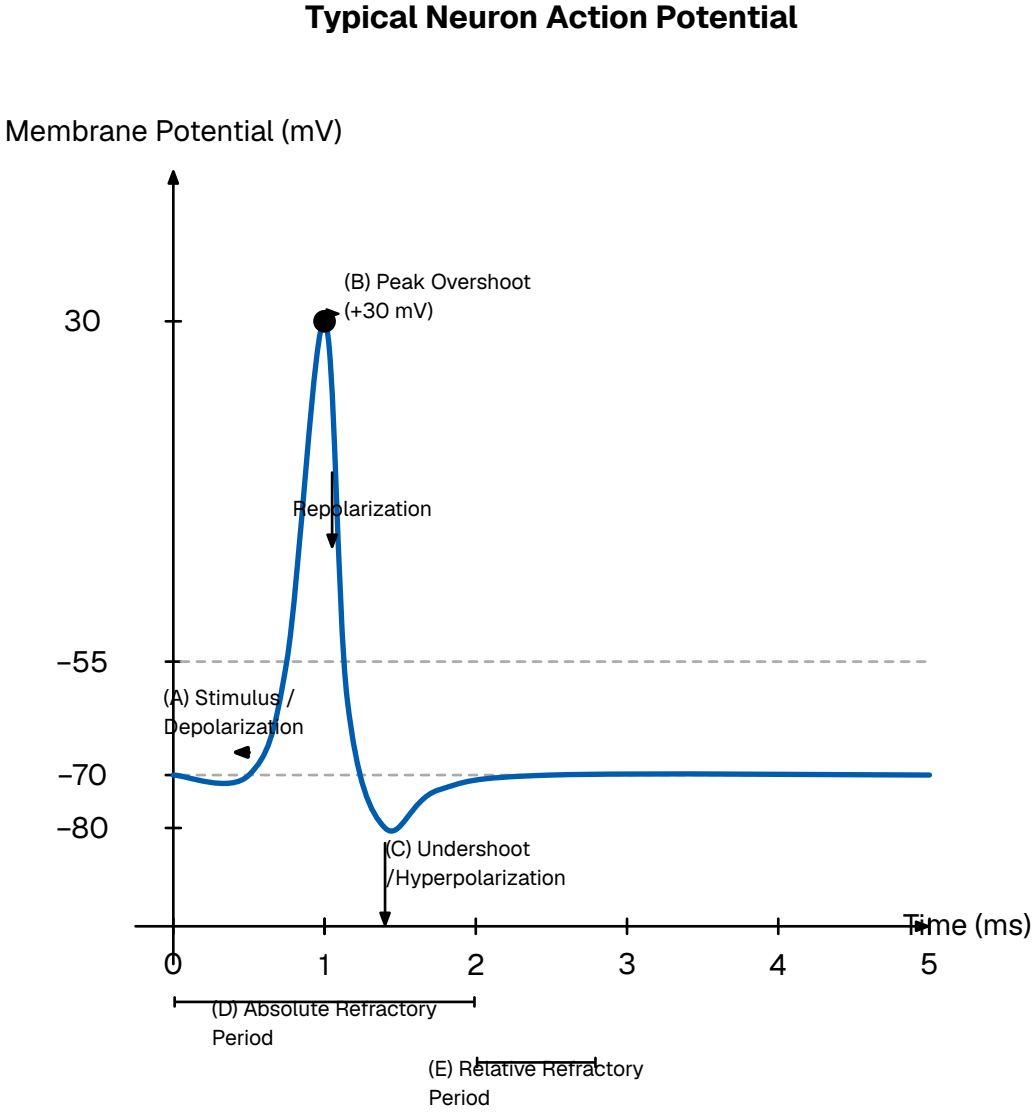


Figure 5: The phases of a typical neuronal action potential. (A) An incoming stimulus depolarizes the membrane past the threshold (−55 mV), triggering a rapid spike. (B) The membrane potential reaches a peak overshoot (+30 mV) before repolarizing. (C) A temporary undershoot (hyperpolarization) occurs before returning to the resting state (−70 mV). The neuron cannot fire during the absolute refractory period (D) and requires a stronger stimulus to fire during the relative refractory period (E).

This biological invariance permits a fundamental simplification in neuromorphic modeling:

> If the spike waveform is invariant across neurons and time, the waveform itself carries no information. Consequently, the information content of the signal is encoded entirely in the precise timing of the spike.

To model this mathematically, we abstract the continuous biophysical voltage trace into a dimensionless point process. We treat the action potential not as a function of voltage over time, but as a singular event occurring at a precise instant, t_f, with

negligible duration. The standard tool for this abstraction is the Dirac delta function denoted as, $\delta(t)$.

The Dirac delta is a generalized distribution defined by the property that it is zero everywhere except at the origin, yet integrates to unity. This represents an idealized pulse of infinite height and zero width, containing a finite unit of effect.

$$\delta(t) = \begin{cases} \infty \text{ if } t = 0 \\ 0 \text{ if } t \neq 0 \end{cases}, \quad \int_{-\infty}^{+\infty} \delta(t)\,\mathrm{d}t = 1$$

Equation 1: The defining properties of the Dirac delta function.

Under this formalism, the output of a neuron is modeled not as a continuous signal, but as a "spike train"—a temporal sequence of these Dirac impulses. For a neuron emitting N spikes at times $\{t^{(1)}, t^{(2)}, ..., t^{(N)}\}$, the output signal $S(t)$ is defined as:

$$S(t) = \sum_{f=1}^{N} \delta\left(t - t^{(f)}\right)$$

Equation 2: A spike train represented as a sum of Dirac delta functions.

This abstraction allows the post-synaptic effect to be modeled using linear systems theory. In neuron models that use this framework, the interaction is treated as instantaneous charge deposition: the arrival of a delta function $\delta\left(t - t_f\right)$ imparts a discrete step-change to the post-synaptic current. This mimics the rapid opening of ion channels without requiring the computational overhead of simulating the complex voltage trajectory.

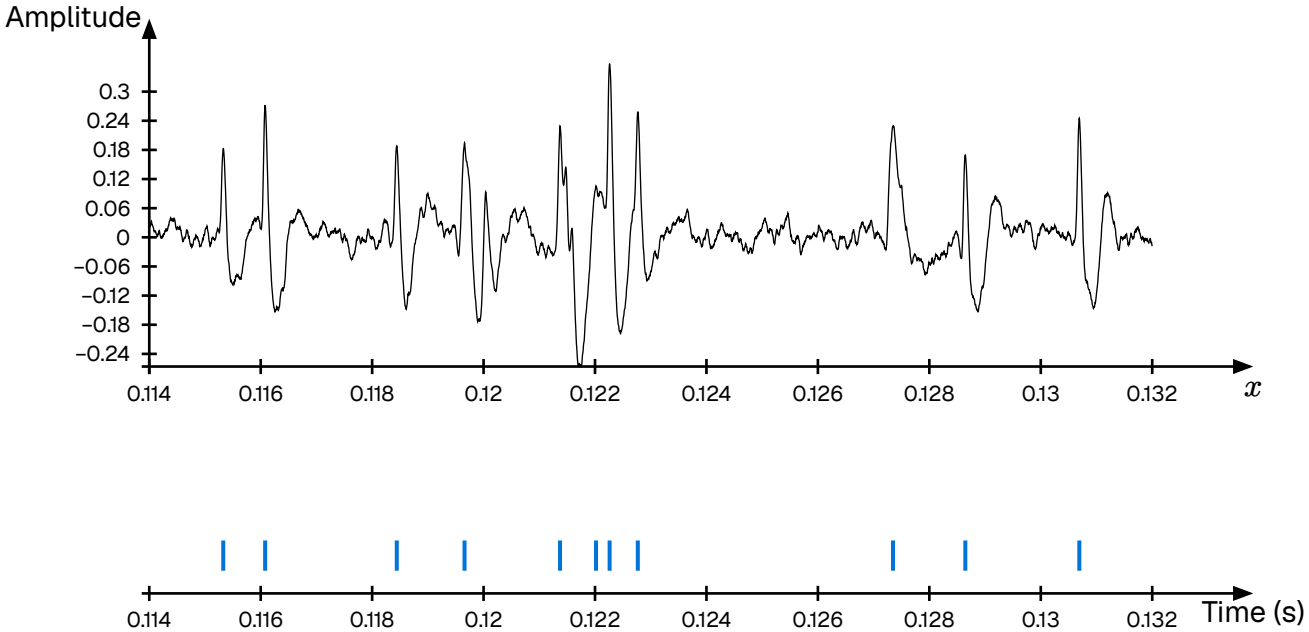


Figure 6: Transformation of continuous membrane voltage (top) into a discrete spike train (bottom).

The shift from continuous values to discrete spike trains fundamentally alters the computational paradigm, moving from spatial representations (magnitude-based) to spatio-temporal representations (time-based).

2.2.3 • NEURON MODELS

In the quest to simulate the brain, there exists a fundamental trade-off between biological realism and computational efficiency. At the high end of the spectrum lie conductance-based models, most notably the Hodgkin-Huxley model. This formalism describes the neuron not as a simple computational unit, but as an electrical circuit with variable resistors representing the precise, non-linear opening and closing dynamics of specific ion channels (sodium, potassium, leak) [1].

Large-scale initiatives, such as the Blue Brain Project, utilize even more granular "multi-compartment" models. These simulations treat the neuron as a complex 3D structure, discretizing the dendritic arbor and axon into hundreds of segments to model how current flows through the specific morphology of the cell [1]. While invaluable for pharmacological research, these models are computationally prohibitive for large-scale neuromorphic engineering. Simulating a mere second of biological time for a small network using these equations requires supercomputing resources.

To build practical, scalable neuromorphic hardware, we must abstract these biophysical details into a phenomenological model. We seek a mathematical framework that captures the essential computational properties—integration, leakage, and thresholding—without simulating the underlying molecular physics.

THE LEAKY INTEGRATE-AND-FIRE (LIF) MODEL

The standard approximation used in neuromorphic engineering is the Leaky Integrate and Fire (LIF) model. This framework aligns perfectly with the "point process" abstraction established in the previous section, as it treats action potentials as instantaneous, discrete events. Its state is defined by a single scalar variable, the membrane potential $u(t)$. The sub-threshold dynamics are governed by a linear differential equation analogous to a simple RC (Resistor-Capacitor) circuit:

$$\tau_m \frac{\mathrm{d}u}{\mathrm{d}t} = -(u - u_{\text{rest}}) + RI(t)$$

Equation 3: The Leaky Integrate-and-Fire (LIF) differential equation. The change in voltage is driven by the leak (decay to rest) and the input current.

Where τ_m is the membrane time constant (determining how fast the neuron "forgets"), u_{rest} is the resting potential, R is the membrane resistance, and $I(t)$ is the input current.

Connecting this to the spike train abstraction derived in the previous section, the input current I(t) is not continuous. It is a sequence of discrete events arriving from pre-synaptic neurons j with weight w_j. Mathematically, this is modeled as a sum of Dirac delta functions:

$$I(t) = \sum jw_j \sum f\delta\left(t - t_{j(f)}\right)$$

Equation 4: Synaptic input modeled as a weighted sum of Dirac delta functions.

Because the differential equation is linear below the threshold, we can solve it analytically. The membrane potential $u(t)$ becomes a convolution of the input spike train with the system's impulse response (a decaying exponential kernel). This means the potential at any moment is simply the sum of the decaying traces of all past spikes:

$$u(t) = u_{\text{rest}} + \sum jwj \sum f \exp\left(-\frac{t - t_{j(f)}}{\tau_m}\right)$$

Equation 5: The analytical solution for the membrane potential. The current voltage is the superposition of all past inputs, decayed by time constant τ_m.

The differential equation above describes the continuous sub-threshold dynamics. To complete the model, we must define the discrete "Fire" mechanism. The LIF neuron operates as a hybrid dynamical system: it integrates continuously until a discontinuity is triggered.

When the membrane potential $u(t)$ exceeds a specific threshold voltage ϑ, the neuron emits a spike (a Dirac delta event). Immediately following this event, the membrane potential is not governed by the differential equation but is forced to a reset value, u_{reset}.

To mimic the biological limit on firing frequency, the model enforces an absolute refractory period, Δ_{ref}. After a spike occurs at time t_f, the neuron is clamped to the resting potential for a duration of Δ_{ref}. During this interval, the differential equation is suspended; the neuron ignores all inputs and cannot fire, regardless of the stimulus intensity.

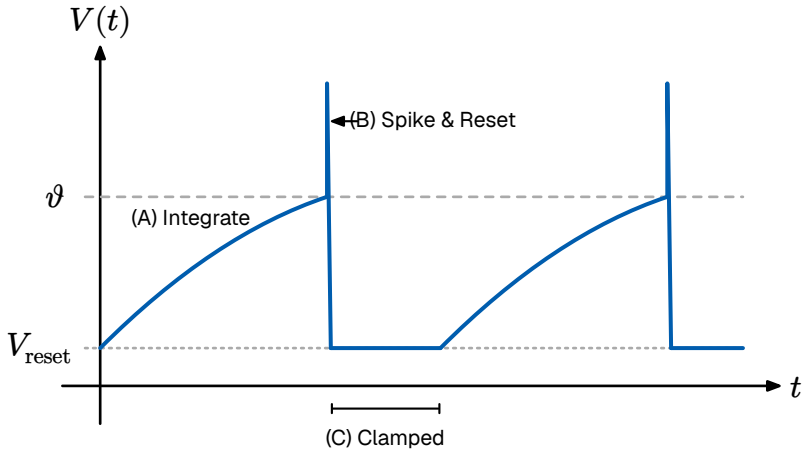


Figure 7: The dynamics of an LIF neuron. (A) The membrane potential integrates inputs. (B) Upon crossing the threshold ϑ, a spike is emitted and the voltage is reset. (C) The voltage is clamped during the refractory period.

Mathematically, this firing condition is expressed as:

$$\text{If } \; u(t) > \vartheta \rightarrow \begin{cases} \text{Emit spike:} \;\; S(t) \rightarrow S(t) + \delta(t) \\ \text{Reset voltage:} \;\; u(t) \rightarrow u_{\text{reset}} \\ \text{Pause integration for} \;\; t \in \left(t_f, t_f + \Delta_{\text{ref}}\right] \end{cases}$$

Equation 6: The discrete firing and reset condition.

This equation represents the engine of most neuromorphic algorithms. It defines a system that integrates information over time and leaks it to ensure temporal relevance. However, once $u(t)$ crosses a threshold ϑ, the linearity breaks and the neuron emits a spike and $u(t)$ is manually reset to u_{reset}.

THE GENERALIZED (ADAPTIVE) LIF MODEL

While the standard LIF model is efficient, it is a one-dimensional system. Its state is strictly determined by $u(t)$. Consequently, it supports only a limited set of firing modes, primarily tonic spiking (regular firing under constant input). It cannot replicate complex non-linear behaviors observed in the cortex, such as bursting (clusters of rapid spikes followed by silence) or spike-frequency adaptation (slowing down after sustained activity).

To capture these dynamics without reverting to the heavy Hodgkin-Huxley equations, we employ the Generalized Leaky Integrate and Fire (GLIF) model. This extends the system by introducing a second state variable, $w(t)$, often called the adaptation variable.

$$\tau_m \frac{\mathrm{d}u}{\mathrm{d}t} = -(u - u_{\text{rest}}) + RI(t) - w \tau_w \frac{\mathrm{d}w}{\mathrm{d}t} = a(u - u_{\text{rest}}) - w$$

Equation 7: The Adaptive GLIF system. The adaptation variable w provides negative feedback, enabling complex dynamics like bursting and adaptation.

In this coupled system, w acts as a negative feedback loop. Every time the neuron spikes, w is incremented by a constant b, which acts as a drag or fatigue on the membrane potential. This simple addition allows the model to exhibit sophisticated neuro-computational properties, bridging the gap between silicon efficiency and biological complexity.

The introduction of the second state variable w fundamentally alters the mathematical nature of the model. While the standard LIF is a one-dimensional system that simply moves toward a threshold, the GLIF constitutes a two-dimensional dynamical system. This increased dimensionality allows the model to exhibit distinct bifurcations—qualitative changes in topological behavior that occur as the input current parameter is varied.

From a dynamical systems perspective, the computational repertoire of a neuron is defined by the type of bifurcation that transitions it from a resting state (fixed point) to a spiking state (limit cycle). The GLIF formulation supports both primary classes of neuronal excitability:

- **Class I (Saddle-Node on Invariant Circle)**: In this regime, the neuron can fire at arbitrarily low frequencies. The frequency-current ($f - I$) curve is continuous, meaning the firing rate increases smoothly from zero as the input current increases. These neurons act as "Integrators," essentially converting the amplitude of the input

signal into a proportional firing frequency. The standard LIF model is restricted exclusively to this class.

- **Class II (Andronov-Hopf Bifurcation)**: In this regime, the onset of repetitive firing is discontinuous. The neuron cannot fire at low rates; as the input current exceeds a critical value, the system jumps immediately to a specific non-zero firing frequency. These neurons often exhibit sub-threshold oscillations before firing, effectively acting as "Resonators." They preferentially respond to inputs matching their intrinsic resonant frequency while filtering out non-resonant signals.

By adjusting the coupling parameters between the membrane potential u and the recovery variable w, the GLIF model can be tuned to operate in either regime. This flexibility allows a single mathematical model to emulate diverse biological behaviors, from the simple integration of sensory transducers to the complex oscillatory synchronization found in cortical interneurons.

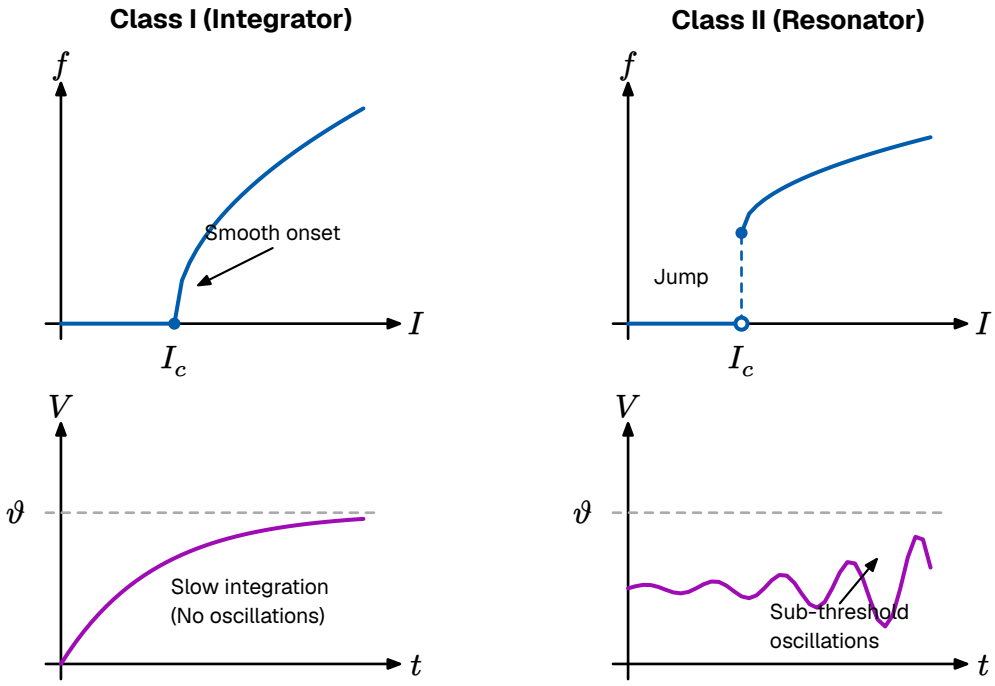


Figure 8: The two primary firing modes determined by bifurcation dynamics. (Left) Class I Integrators show a smooth frequency response. (Right) Class II Resonators (Hopf) show a discontinuous jump and sub-threshold oscillations.

It is natural to question whether such a mathematically reduced model can genuinely capture the behavior of biological neurons. While the GLIF model discards the specific ionic mechanisms of the Hodgkin-Huxley equations, empirical validation suggests that it retains the essential computational dynamics.

In the 2008 *Quantitative Single-Neuron Modeling Competition* [1] organized by the INCF, various models were tested on their ability to predict the precise spike times of real cortical neurons recorded in vitro. Unexpectedly, simple phenomenological models like the Generalized LIF (specifically the Adaptive Exponential Integrate-and-Fire, or AdEx) outperformed highly detailed biophysical models.

The reason for this counter-intuitive success is parameter sensitivity. Complex conductance-based models have dozens of unobservable parameters (channel densities, pore open probabilities) that are difficult to tune. In contrast, the GLIF model captures the "net effect" of these mechanisms—integration, thresholding, and adaptation—using macroscopic parameters that can be robustly fitted to data [1].

As demonstrated by Izhikevich (2003), this simple system of two differential equations is capable of reproducing all known firing patterns observed in the mammalian cortex, including regular spiking, intrinsic bursting, and chattering [1].

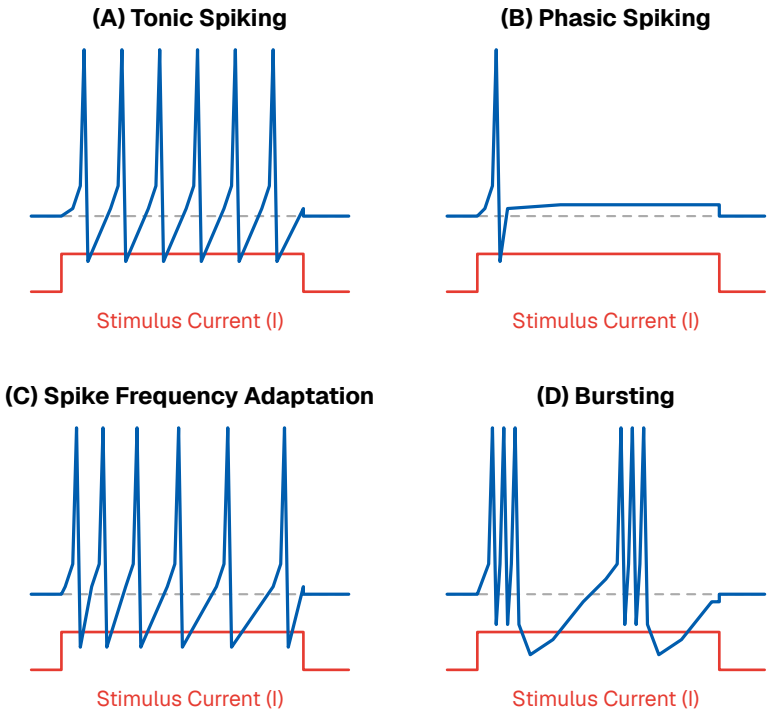


Figure 9: The Generalized LIF model is capable of reproducing the diverse firing patterns of biological cortical neurons, as categorized by Izhikevich (2003) [1].

Consequently, for the purpose of neuromorphic engineering—where the goal is to emulate the computation (the timing of information processing) rather than the chemistry—the GLIF model represents a good trade-off between fidelity and efficiency.

2.2.4 • NEURAL CODING

In classical digital computing, information is represented by combining bits into richer structures, such as floating-point numbers. For instance, the luminance of a pixel is typically stored as a discrete 8-bit or 32-bit integer. Conversely, analog electronics represent values as continuous currents or voltages, offering infinite resolution within the dynamic range of the hardware.

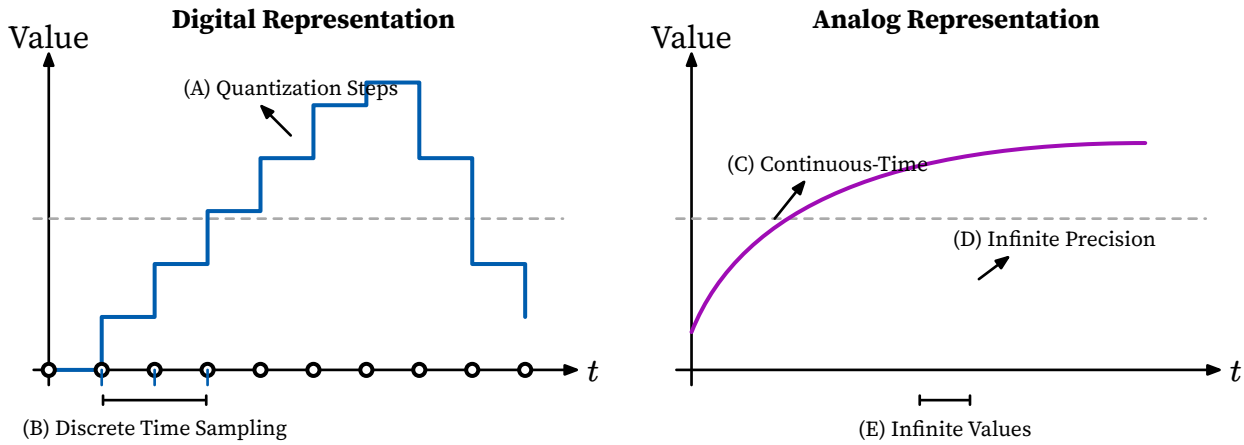


Figure 10: Digital left analog right representation

The biological brain occupies a unique middle ground. While neurons operate using analog membrane potentials, their communication output—the action potential—is discrete and binary. As established in Section 2.2.2, the waveform of a spike is stereotypical; it looks like a "digital bit" in amplitude. However, unlike a digital computer which is synchronized to a rigid clock, these spikes occur in continuous time. Therefore, the information in the nervous system is not stored in the shape of the signal, but in the structure of the spike train itself.

Deciphering the "Neural Code"—the set of rules by which sensory stimuli are translated into these spike sequences—remains one of the central challenges in neuroscience. Currently, several coding schemes are hypothesized to coexist, each offering different trade-offs between latency, information density, and robustness.

RATE CODING

The most traditional interpretation of neural activity is rate coding. In this paradigm, information is conveyed by the mean firing frequency of a neuron over a specific temporal window. A strong stimulus (e.g., high pressure on skin) elicits a high firing rate, while a weak stimulus results in sparse activity.

This model effectively treats the neuron as an Analog-to-Digital converter where the precise timing of individual spikes is treated as noise; only the average count carries the signal. While rate coding is robust and easily observed in motor neurons, it suffers from a fundamental latency barrier. To estimate a rate with reasonable precision, the post-synaptic neuron must integrate spikes over a significant duration (tens or hundreds of milliseconds). This contradicts the rapid reaction times (often < 100 ms) observed in biological agents, suggesting that rate coding alone cannot account for time-critical processing.

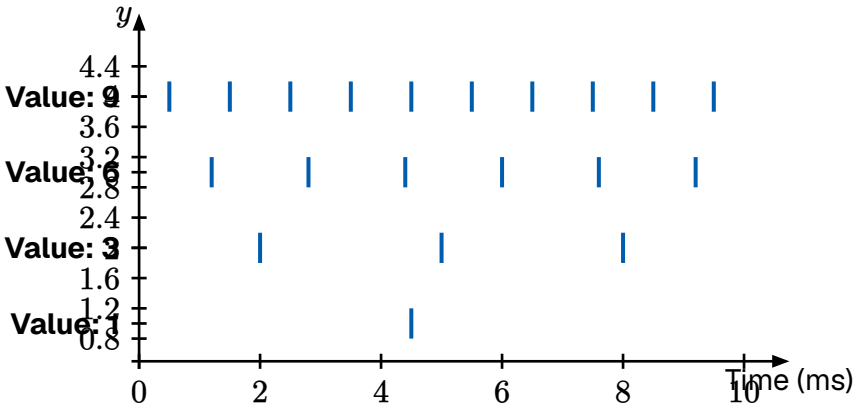


Figure 11: Rate Coding: The stimulus intensity is encoded in the frequency of the spike train. Stronger stimuli elicit more spikes per second.

TEMPORAL CODING

To explain the speed of biological processing, neuromorphic engineering emphasizes temporal coding. In this regime, the precise timing of a spike carries significant information. A primary example is Time-To-First-Spike (TTFS) coding.

In a TTFS scheme, the intensity of a stimulus is inversely mapped to the latency of the response relative to a stimulus onset. A stronger input causes the neuron to integrate and cross the threshold faster, firing earlier than neurons receiving weak inputs. This shifts the computational model from counting spikes to a "race" between spikes.

In a network utilizing lateral inhibition (Winner-Take-All (WTA)), the first neuron to fire inhibits its neighbors, allowing a decision to be made as soon as the first meaningful bit of data arrives. This eliminates the need to wait for a time window to close, drastically reducing latency. Furthermore, since TTFS coding prioritizes the strongest signals, it acts as a natural filter: the most prominent features arrive first, allowing the system to process signal over noise.

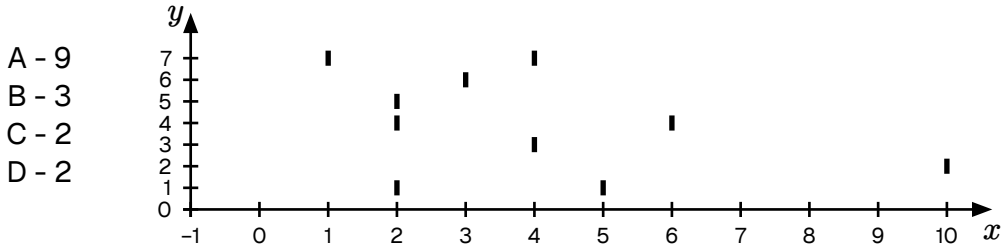


Figure 12: Temporal Coding (TTFS): Stimulus intensity is encoded in the latency of the response. Stronger inputs (I_1) trigger an earlier spike (t_1) compared to weaker inputs (I_2).

THE PHASE AMBIGUITY PROBLEM

A critical challenge in temporal coding is the need for a temporal reference frame. In Rate Coding, the "phase" (absolute timing) is irrelevant. However, in Temporal Coding, a spike at time t only has meaning relative to a reference t_0. If the receiver does not know when the stimulus started, it cannot decode the latency.

In engineering, this is solved by a global clock or a “frame start” signal. In the brain, evidence suggests that background oscillatory rhythms (brain waves, such as theta or gamma cycles) may serve as this global reference, allowing populations of neurons to synchronize their “clocks” and decode relative timings accurately.

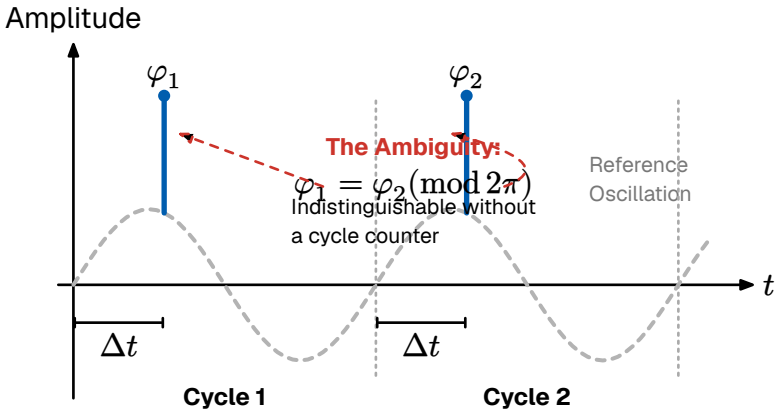


Figure 13: The phase ambiguity problem in temporal encoding. Spikes occurring at the same relative phase (φ_1 and φ_2) across different oscillation cycles are mathematically indistinguishable ($\varphi_1 = \varphi_2 (\mathrm{mod}\, 2\pi)$). Without a mechanism to track the global cycle count, downstream neurons cannot determine whether a spike represents a delayed response to a previous stimulus or an early response to a new one.

POPULATION & SPARSE CODING

While single-neuron codes provide the basic signaling mechanism, the brain employs ensemble strategies to ensure robustness and precision. Crucially, population coding is not an alternative to rate or temporal coding; rather, it functions as a higher-order representation that aggregates these signals.

In population coding, variables are represented by the joint activity of a large ensemble of neurons. A classic example is found in the Primary Visual Cortex (V1), where neurons are tuned to specific edge orientations. A single neuron might respond maximally to a vertical bar (90°), but will also fire weakly for 80° or 100°.

Relying on a single neuron introduces ambiguity: a weak response could indicate a perfect stimulus at low contrast, or an imperfect stimulus at high contrast. The brain resolves this by reading the population vector—the weighted average of activity across the entire local group. By combining the noisy, broad tuning curves of many neurons, the network can reconstruct the stimulus orientation with a precision far greater than that of any individual cell.

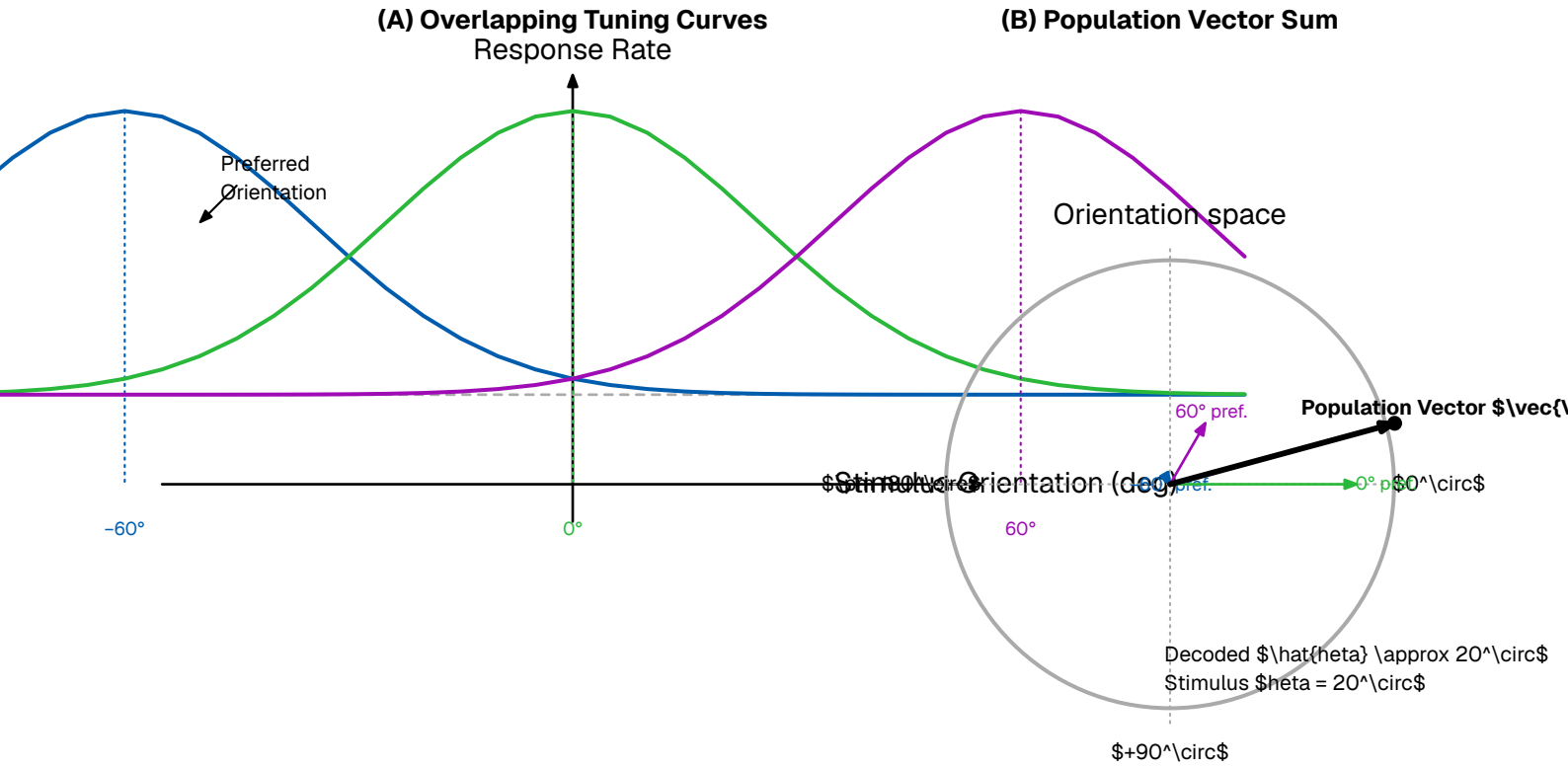


Figure 14: Population Coding in the Visual Cortex. (A) Individual neurons have broad "tuning curves" centered on a preferred orientation. (B) The precise stimulus angle is recovered by decoding the population vector sum.

Furthermore, the brain optimizes for metabolic efficiency through sparse coding. This theory posits that neural systems minimize the number of active neurons required to represent a stimulus. At any given moment, out of billions of neurons, only a tiny fraction are firing. This stands in contrast to "dense" coding (where many units participate) or "local" coding (the hypothetical "grandmother cell" where one unique unit represents one unique object). Sparse coding strikes a mathematical balance: it creates a representation that has a high capacity for information but consumes minimal energy.

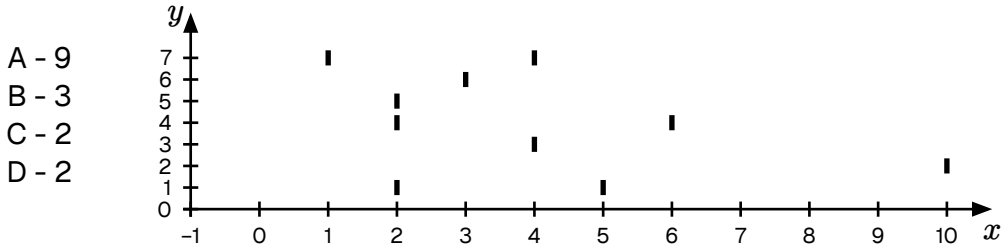


Figure 15: Comparison of coding densities. Sparse coding (right) activates a minimal subset of neurons to represent a feature, maximizing energy efficiency compared to dense coding.

COEXISTENCE OF CODES

It was historically theorized that the brain utilizes a single universal code. However, modern evidence suggests that these schemes are not mutually exclusive but rather complementary [1]. A neural circuit may utilize TTFS for the initial rapid response

(alerting the system to a change) and transition to rate coding to maintain a sustained muscle contraction or represent a static value.

Neuromorphic systems often adopt a hybrid approach: using temporal codes for the energy-efficient transmission of sparse sensory events, and rate-based readouts for interfacing with standard control systems or actuators.

2.2.5 • NEURAL NETWORKS

Having established the mathematical description of the individual neuron, we now turn to the collective behavior of these units. A single neuron, regardless of its dynamical complexity, is of limited computational utility in isolation. Functional intelligence emerges only when these units are organized into specific structural topologies.

The brain is not a random mesh of connections; it is constructed from recurring architectural "motifs" that appear across various cortical areas. Understanding these motifs is essential for designing neuromorphic systems that transcend simple feed-forward processing.

SYNAPTIC EFFICACY & WEIGHTS

Before analyzing the structural topology of networks, we must define the fundamental unit of connectivity: the synapse. In the biological brain, neurons do not touch; they are separated by a microscopic gap known as the synaptic cleft. Communication across this gap is chemical, mediated by the release of neurotransmitters.

The efficiency of this transmission—determined by factors such as the amount of neurotransmitter released and the number of post-synaptic receptors—is abstracted in mathematical models as the synaptic weight (w).

In the Spiking Neural Network (SNN) formalism, the weight represents a scaling factor for the incoming spike. When a pre-synaptic neuron j fires a spike at time t_j, it induces a Post Synaptic Current (PSC) in neuron i scaled by the weight w_{ij}. Mathematically, the total synaptic input $I(t)$ is the weighted sum of all incoming spike trains:

$$I_{i(t)} = \sum_{j} w_{ij} \cdot S_{j(t)}$$

Equation 8: The synaptic input current as a weighted sum of incoming impulses.

Synaptic weights determine not just the magnitude but also if the synapse is excitatory or inhibitory.

- **Excitatory Synapses ($w > 0$):** These depolarize the target neuron, pushing its membrane potential closer to the firing threshold (e.g., Glutamate synapses).

- **Inhibitory Synapses ($w < 0$):** These hyperpolarize the target neuron, pushing the potential away from the threshold (e.g., GABA synapses).

A fundamental constraint in biological networks, known as Dale's Principle, states that a neuron performs the same chemical action at all of its synaptic outputs. This means a neuron is strictly excitatory or strictly inhibitory; it cannot send positive signals to one neighbor and negative signals to another. While standard ANNs often violate this rule for mathematical convenience (allowing weights to flip signs during training), bio-plausible neuromorphic architectures often enforce this constraint to mimic the distinct populations of Pyramidal (excitatory) and Interneuron (inhibitory) cells found in the cortex.

The network must maintain a precise Excitation-Inhibition (E/I) Balance. The brain operates at a critical point of instability:

- **Excess Excitation** leads to runaway feedback loops (analogous to epileptic seizures).
- **Excess Inhibition** leads to signal extinction (quiescence).

DIRECTIONALITY

Structurally, neural topologies can be categorized by the flow of information.

In sensory peripheries (such as the retina) and early processing stages, information flows unidirectionally from input to output. This topology supports rapid, reflex-like feature extraction. This configuration is known as a feed-forward network, which is mathematically equivalent to a Directed Acyclic Graph (Directional Acyclic Graph (DAG)) and serves as the standard architecture for most Deep Learning CNNs.

In higher cognitive areas, the dominant topology is recurrence. Neurons form feedback loops, connecting back to themselves or to distinct layers. This recurrence introduces a time component to the computation, transforming the network into a dynamical system where the current output depends not only on the input but on the network's previous state (history).

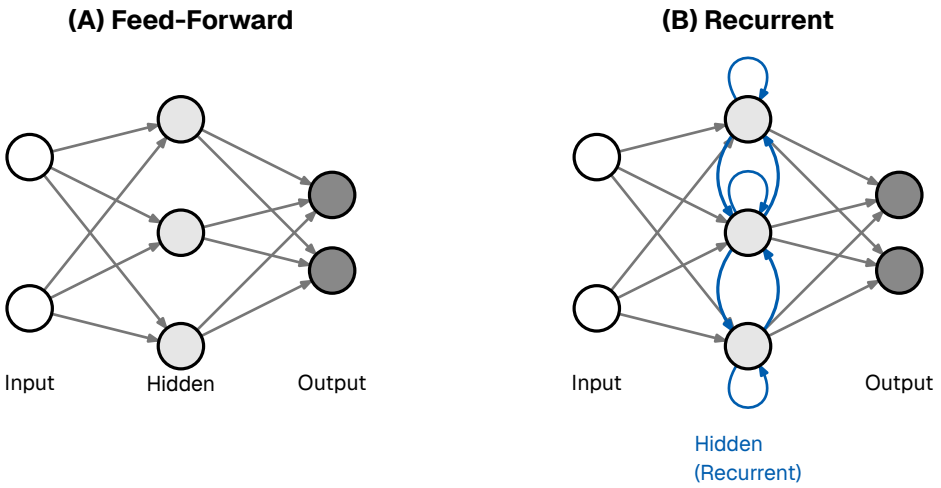


Figure 16: Network topologies. (A) Feed-Forward. (B) Recurrent.

THE SYNAPTIC HYPOTHESIS: STRUCTURE AS FUNCTION

It is a fundamental premise in computational neuroscience that the neuron operates largely as a generic processing unit. While distinct neuronal subtypes exist, a pyramidal neuron in the visual cortex operates on electrophysiological principles identical to those of a neuron in the motor cortex. Consequently, the functional identity of a neural circuit—the specific architecture that determines whether a network processes visual stimuli or governs motor control—is derived principally from the topology and synaptic efficacy of its interconnections.

This paradigm, known as the Synaptic Hypothesis, posits that the physical configuration of synaptic weights (the "Connectome") constitutes the substrate for all long-term memory and learned skills. Unlike artificial systems where data is distinct from the processor, biological memory is realized as a specific configuration of the structural connections themselves. Therefore, the formation of a new memory necessitates a physical alteration of the neural "hardware."

Philosophically, this architecture challenges the ontological distinction between "data" and "program." In a synaptic system, a memory is not a static artifact retrieved from a repository, but a latent computational potential. To "remember" is not to read, but to run; the retrieval of a memory is the active reconstruction of a neural state based on the compressed statistical regularities encoded in the weights. Thus, every act of memory retrieval is inherently a computational process—a simulation regenerated on demand rather than a file accessed from storage.

This dynamic nature implies that memory is labile; because retrieval is a constructive process, activating a memory trace renders it temporarily malleable, allowing for reconsolidation—the updating of old memories with new context. Furthermore, this distributed storage ensures graceful degradation; unlike a digital file that becomes unreadable if a segment is corrupted, a synaptic memory persists as a robust statistical correlation, fading in resolution rather than failing catastrophically under damage.

It must be noted, however, that the brain is not a uniform blank slate at creation. Critical functional systems, such as the visual and motor cortices, are initialized with a conserved topology during embryonic development. All humans share these fundamental circuits; postnatal learning in these regions is characterized not only by weight adjustment but by critical periods of experience-dependent pruning and refinement, rather than the construction of architecture from the ground up.

INHIBITION PATTERNS

A ubiquitous micro-circuit motif in the cortex is lateral inhibition. In this configuration, an active excitatory neuron stimulates distinct inhibitory interneurons, which in turn suppress the activity of neighboring excitatory neurons. This competition engenders a WTA dynamic: as one neuron—representing a specific feature or decision—becomes active, it effectively silences its competitors. In the context of neuromorphic engineering, WTA circuits are indispensable; they provide a physical mechanism for both noise reduction, by actively suppressing weak, sub-threshold signals, and categorical decision making, enabling the circuit to autonomously select the most salient option without the need for a central processor to sort or compare values.

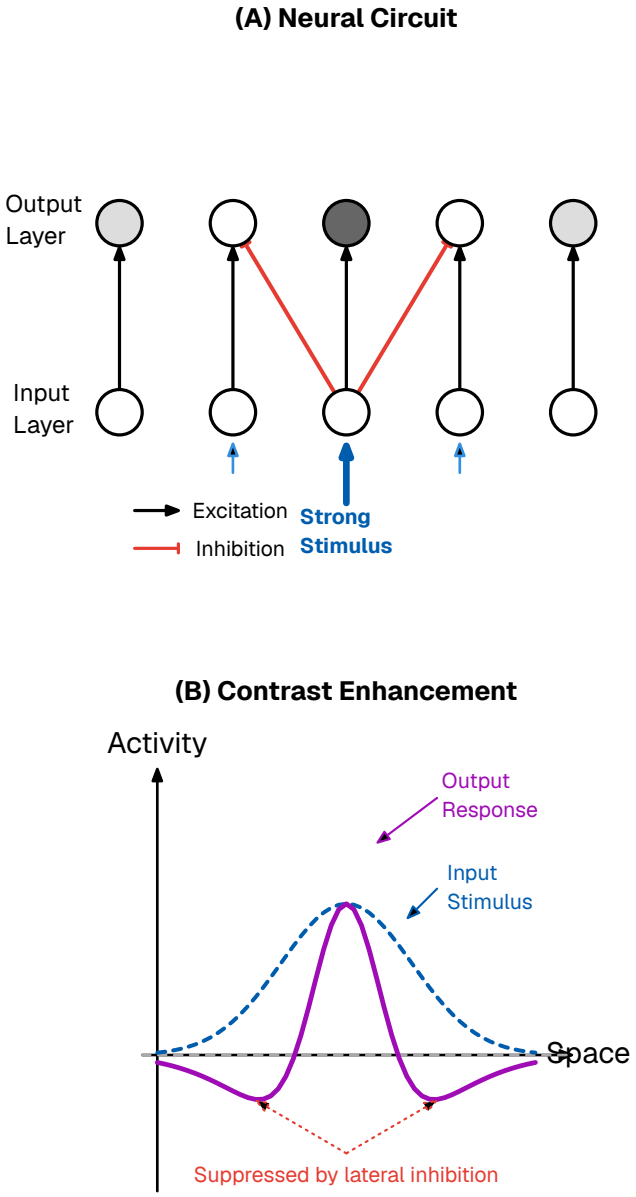


Figure 17: The mechanism of lateral inhibition. (A) A highly stimulated neuron in the input layer strongly excites its corresponding output neuron while simultaneously sending lateral inhibitory signals to its immediate neighbors. (B) This architectural motif acts as a spatial filter, producing a contrast enhancement effect. A broad input stimulus (dashed blue line) is transformed into a sharper output response (solid purple line) characterized by an amplified center and suppressed surroundings (a "Mexican hat" profile), thereby sharpening signal boundaries.

While lateral inhibition processes information in the spatial domain, Feed-Forward Inhibition (FFI) operates in the temporal domain. Structurally, this motif bifurcates an input signal into two parallel pathways: a direct excitatory route to the target neuron, and a disynaptic inhibitory route that reaches the same target with a slight synaptic delay. This architecture creates a narrow "temporal window of opportunity." Because the excitation triggers the neuron immediately before the delayed inhibition abruptly truncates the response, the neuron is prevented from integrating noise over extended durations. Consequently, FFI forces the neuron to function as a precise Coincidence Detector rather than a sluggish integrator, a dynamic that is

fundamental to sound localization in the auditory cortex and fine-grain timing in the somatosensory system.

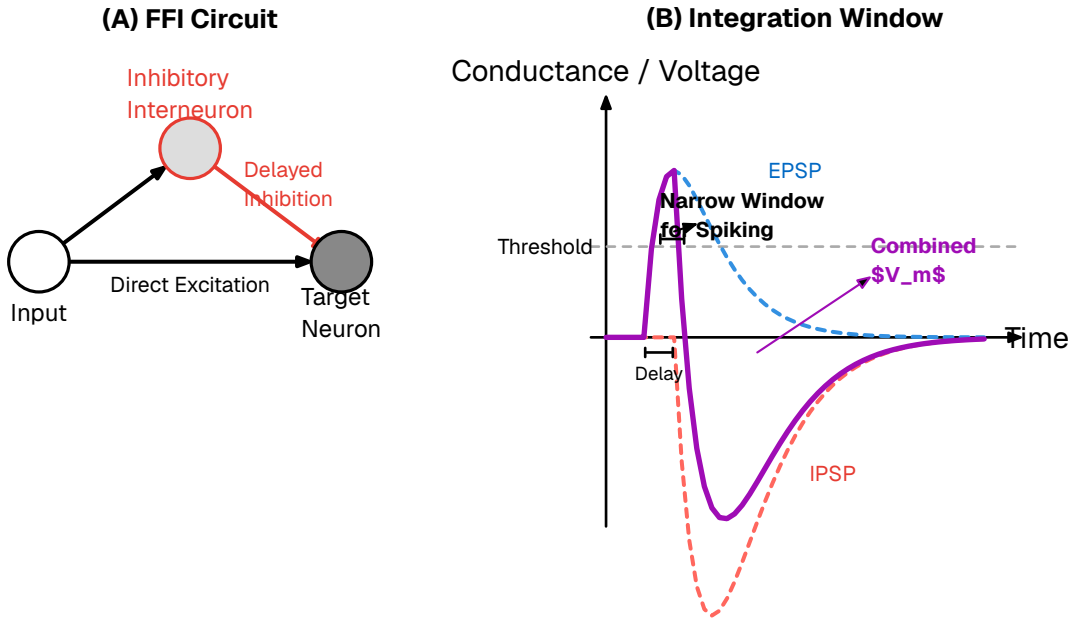


Figure 18: Feed-Forward Inhibition (FFI). The input excites the target but also drives an inhibitor that shuts the target down shortly after. This creates a precise temporal window for firing.

Distinct from the competitive nature of lateral inhibition, Feedback Inhibition functions as a local regulatory loop. In this circuit, an active excitatory neuron recruits an inhibitory interneuron, which subsequently projects back to suppress the original sender. This negative feedback loop serves two critical engineering functions. First, it provides homeostatic Gain Control, dynamically compressing the signal range to prevent neuronal saturation during high-intensity input. Second, the inherent conduction delays within the loop induce rhythmic firing in the population. This mechanism is the primary driver of Gamma frequency (30–80 Hz) oscillations, which are hypothesized to facilitate the synchronization of communication between distant cortical regions.

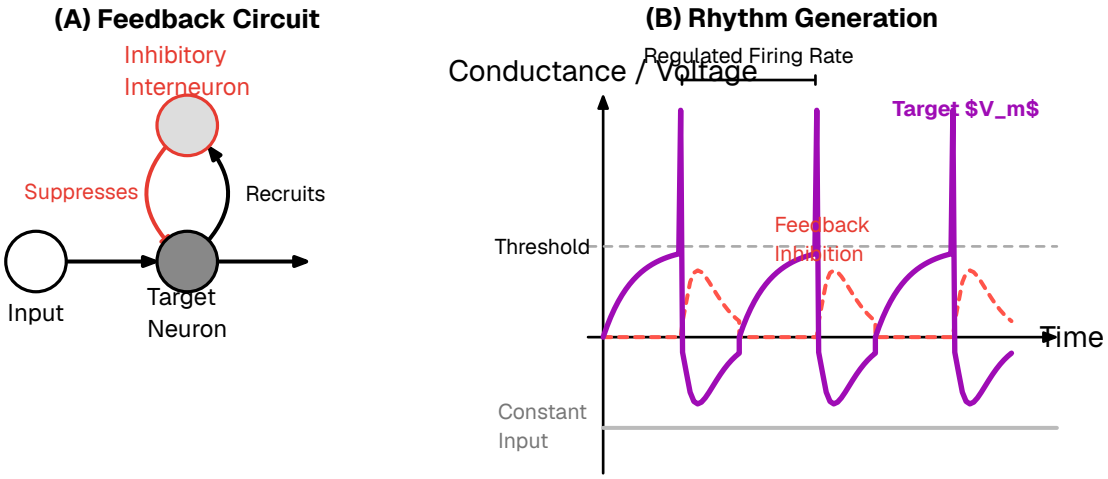
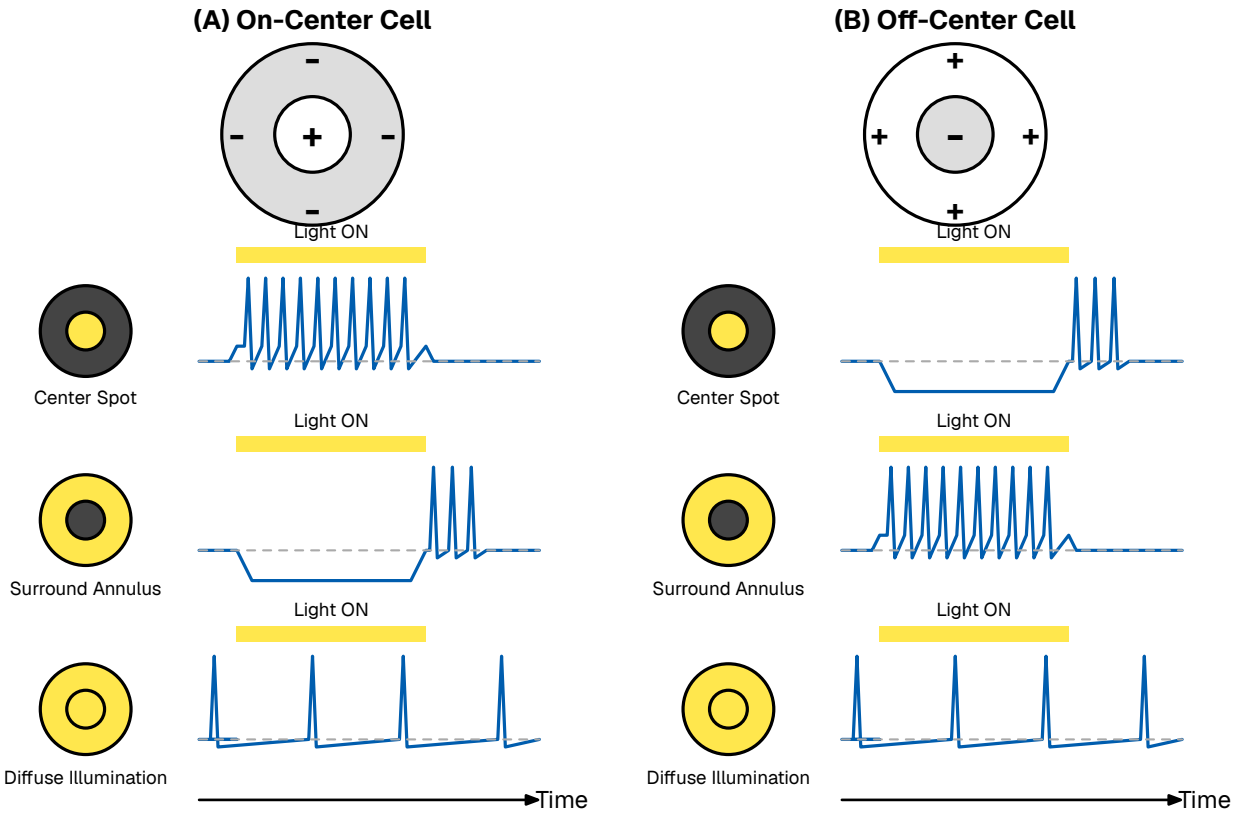


Figure 19: Feedback Inhibition. The active neuron recruits an inhibitor to suppress itself, creating a self-regulating negative feedback loop used for gain control and rhythm generation.

SYSTEM EXAMPLE: THE VISUAL HIERARCHY

The primate visual system serves as the archetypal biological model for neuromorphic architecture. Rather than processing images as static, monolithic frames, the cortex operates as a hierarchical cascade of feature extraction, mathematically approximating a deep Directed Acyclic Graph (DAG). This processing pipeline begins at the sensor level and progressively abstracts data through specialized stages.

Visual processing initiates in the retina, which functions not merely as a passive camera sensor but as a pre-processing neural computer. Photoreceptors connect to Retinal Ganglion Cells (RGCs) via a lateral inhibition architecture known as "Center-Surround." This configuration creates two complementary cell types: On-Center cells, which fire when a bright stimulus is surrounded by darkness, and Off-Center cells, which respond to dark stimuli on bright backgrounds. This antagonist arrangement effectively acts as a hardware-level convolution filter (specifically, a Difference of Gaussians). By discarding redundant background data and transmitting only contrast changes, the retina performs significant data compression and edge enhancement before the signal ever reaches the brain.



Lateral Inhibition acts as an Edge-Enhancement Filter:
Under diffuse illumination (bottom traces), excitatory and inhibitory regions cancel each other out, resulting in weak baseline firing. The cell only fires strongly when a contrast edge selectively illuminates the excitatory region without triggering the inhibitory surround.

Figure 20: Retinal Receptive Fields. (A) On-Center cell firing logic. (B) Off-Center cell firing logic. This lateral inhibition acts as an edge-enhancement filter.

Upon reaching the Primary Visual Cortex (V1), the data undergoes a dimensional transformation from "dots of light" to "geometric primitives." In a mechanism first described by Hubel and Wiesel, the outputs of multiple aligned On/Off-Center cells converge onto single Simple Cells via strong excitatory synapses. This convergence creates a neuron acting as a band-pass Gabor filter, capable of detecting specific spatial frequencies and orientations. Consequently, a V1 neuron does not represent a pixel, but rather a concept: a vertical edge, a 45-degree slant, or a horizontal boundary.

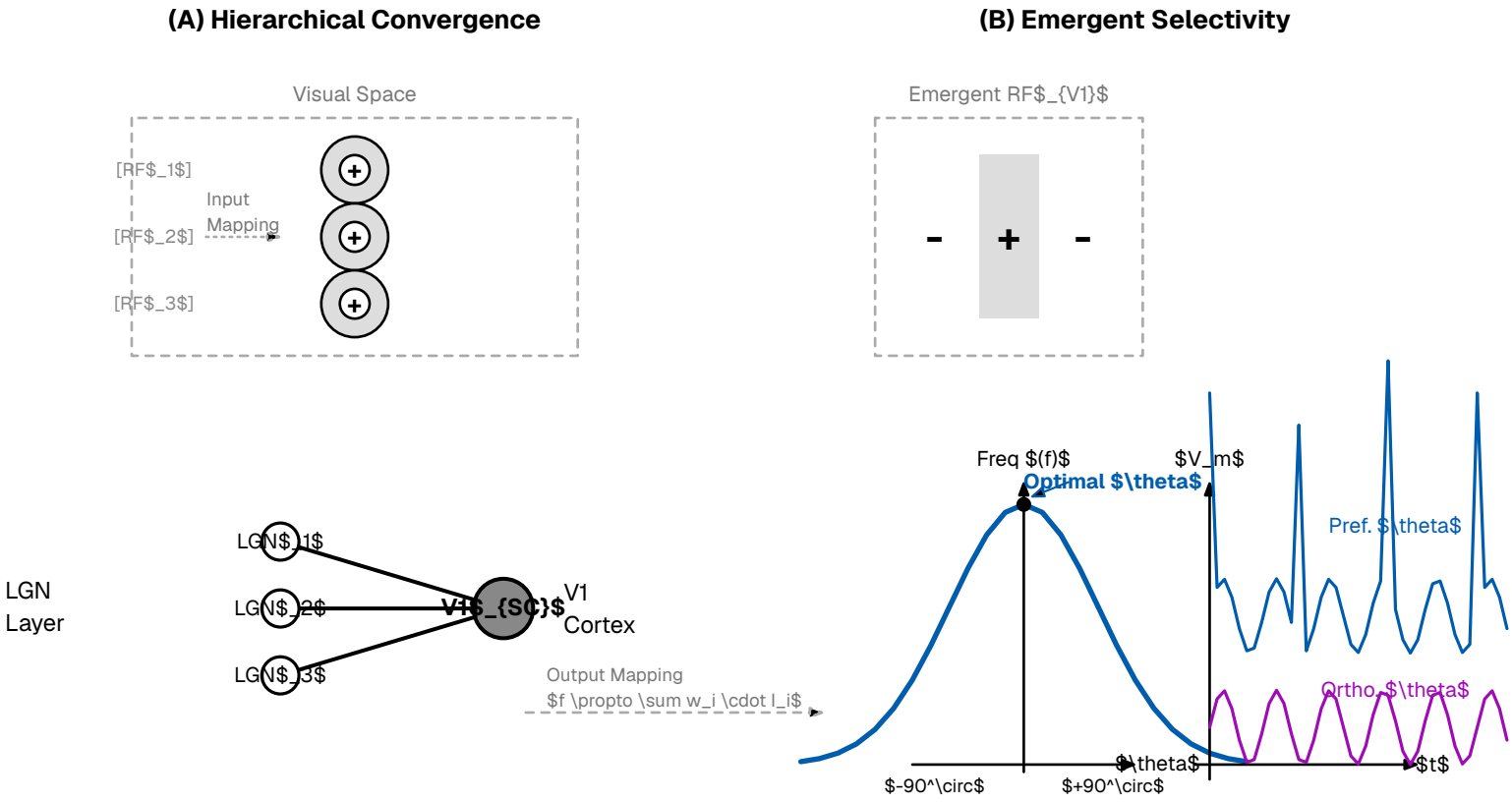


Figure 21: The Hubel & Wiesel Model. Outputs from distinct Retinal/LGN cells converge to form V1 Simple Cells, creating orientation-selective edge detectors.

Following feature extraction in V1, the architecture bifurcates into two distinct, parallel processing streams, known as the Dual-Stream Hypothesis. The Ventral Stream ("What" Pathway) projects to the temporal lobe, functioning as a deep feed-forward network that hierarchically constructs object identity—progressing from V1 edges to V2 textures, V4 shapes, and finally IT Cortex object recognition. Conversely, the Dorsal Stream ("Where" Pathway) projects to the parietal lobe and is specialized for high temporal resolution. Rather than identifying objects, this stream calculates the optical flow and spatial coordinates required to guide motor actions. Modern neuromorphic pipelines, particularly those utilizing event cameras, mimic this biological split to separate the computationally expensive task of object recognition from the latency-critical task of motion tracking.

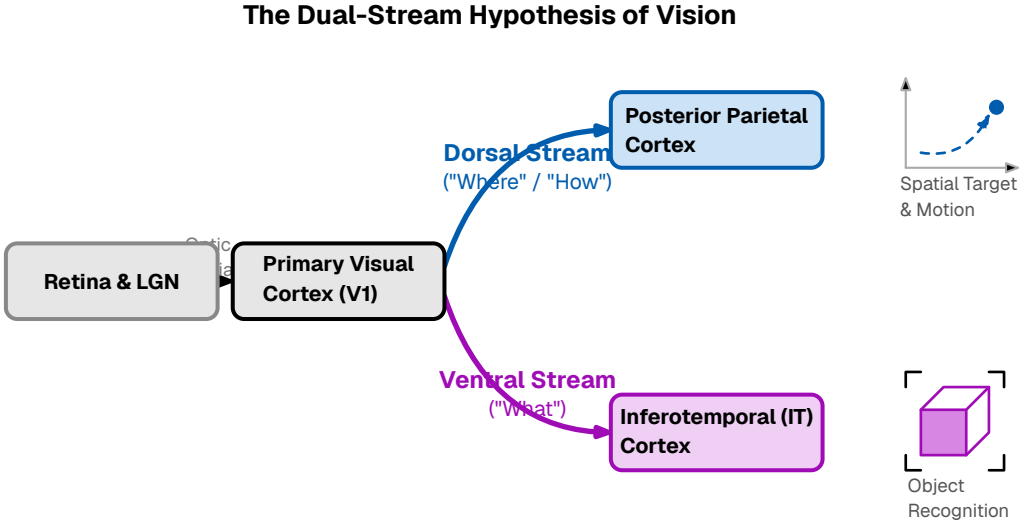


Figure 22: The Dual-Stream Hypothesis. Visual information splits into the Ventral stream for object recognition and the Dorsal stream for spatial navigation.

MACRO-CIRCUIT MOTIFS

While micro-circuit motifs govern local signal processing, the brain's broader computational capabilities—such as working memory and global data integration—emerge from specific large-scale structural organizations. Although the scope of this thesis focuses on fundamental components, understanding these macro-motifs is essential for conceptualizing how neuromorphic systems can scale beyond simple pattern recognition to achieve cognitive reasoning and persistent state maintenance.

At the topological level, biological neural networks differ significantly from both regular lattices and random graphs, exhibiting instead a "Small-World" architecture. In this configuration, nodes form tightly knit local clusters (high clustering coefficient) while simultaneously maintaining sparse "long-range" connections that bridge distant clusters. For neuromorphic engineering, this topology represents a critical optimization problem: it minimizes the physical wiring cost and metabolic overhead while ensuring that the path length between any two nodes remains short. This allows for rapid global synchronization of data without the prohibitive spatial requirements of a fully connected network.

Within the recurrent topology described earlier, the challenge of maintaining information over time without constant external input is solved through Attractor Networks. Mathematically, these circuits function as recurrent dynamical systems where the network's energy landscape contains stable fixed points, or "basins of attraction." When neural activity enters one of these basins, it settles into a self-sustaining pattern that persists even after the stimulus is removed, effectively acting as the biological substrate for Working Memory.

This stability mechanism is what enables Content-Addressable Memory (associative memory). Information is retrieved not by address, but by "keying" the network with a partial pattern (e.g., a familiar scent). This input pushes the system state near a

basin, triggering the circuit to settle into the associated attractor state and effectively re-computing the missing information. These attractors are generally categorized into two types:

- **Discrete Attractors (Point Attractors)**: Used for categorical memory and auto-associative error correction.
- **Continuous Attractors (Line/Ring Attractors)**: Used for encoding continuous variables like spatial orientation and navigation coordinates.

Finally, the physical instantiation of these functions is not realized through a homogeneous mass of neurons, but through a highly modular architecture known as the Cortical Column. The neocortex is organized into discrete, repetitive units, where each column functions as a canonical microcircuit—a specialized "processing core" containing all necessary layers and inhibitory motifs to process a specific receptive field. This modularity suggests that biological intelligence scales not by inventing new, complex algorithms for every task, but by replicating a standard, versatile circuit. In hardware design, this principle is isomorphic to the "tiling" approach seen in modern neuromorphic chips, where a single optimized processing core is replicated thousands of times to achieve massive scalability and fault tolerance.

2.2.6 • BIOLOGICAL LEARNING

As previously established, the functional identity of a neural circuit is not defined by a transient software state, but by its physical hardware configuration. Consequently, "learning" in a biological substrate cannot be viewed as a simple parameter optimization; it is a fundamental morphological process. If structure dictates function, then the acquisition of new skills or memories necessitates the physical restructuring of the connectome itself.

Because the brain lacks a central supervisor or global communication bus, this restructuring must be driven by Locality. A synapse can only change based on information physically available at the cleft: the activity of the pre-synaptic axon, the voltage of the post-synaptic dendrite, and the immediate neurochemical environment. Despite this constraint, the brain successfully credits specific synaptic events with outcomes that occur seconds or minutes later.

This adaptation occurs across multiple timescales and spatial resolutions via two distinct mechanisms: Structural Plasticity (the rewiring of the network topology) and Synaptic Plasticity (the modulation of connection strength).

STRUCTURAL PLASTICITY

While synaptic weight adjustment accounts for rapid learning and pattern recognition, the long-term storage of information and the optimization of energy efficiency are governed by structural plasticity. This mechanism involves the physical gene-

sis (synaptogenesis) and removal (pruning) of synapses and even entire axonal branches.

- **Synaptogenesis**: When neurons are repeatedly co-active but lack a direct connection, the brain can physically grow new dendritic spines and axonal boutons to bridge the gap. This effectively alters the network's topology, creating new pathways for information flow where none existed before.
- **Pruning**: Equally critical is the removal of redundant or noisy connections. During sleep and developmental critical periods, the brain aggressively prunes weak synapses. This "sparsification" reduces metabolic cost and increases the signal-to-noise ratio of the circuit by removing irrelevant pathways.

In the context of the Synaptic Hypothesis, structural plasticity represents the "compiling" of temporary associations into permanent hardware architecture.

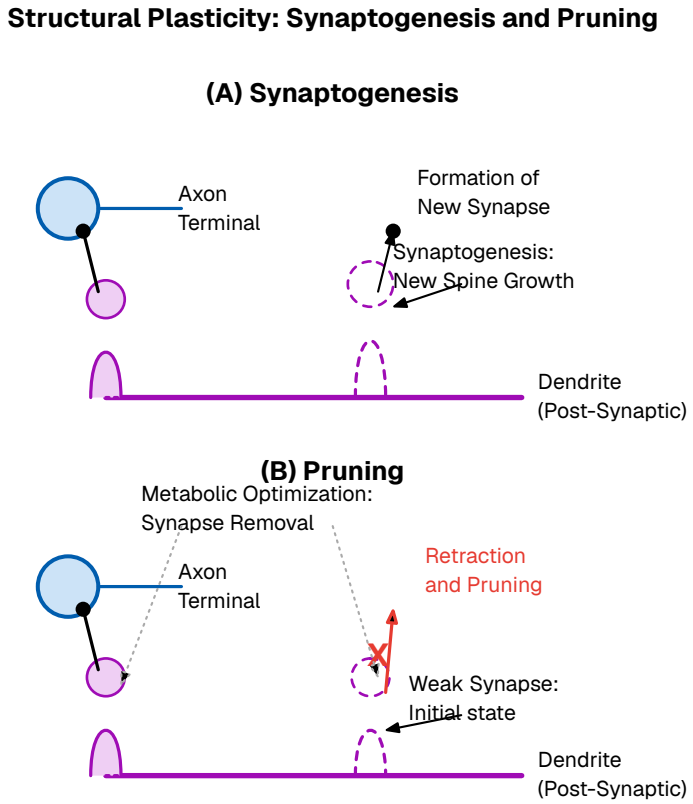


Figure 23: Structural Plasticity. (A) Synaptogenesis: The growth of new dendritic spines to form connections. (B) Pruning: The retraction of unused connections to optimize metabolic efficiency.

SYNAPTIC PLASTICITY

Once a structural connection exists, its efficacy—the magnitude of the post-synaptic response to a pre-synaptic spike—must be tuned. In biological terms, this "weight" corresponds to the amount of neurotransmitter released and the density of receptors on the receiving side. This fine-grained adjustment is governed by local learning rules.

HEBBIAN LEARNING: RATE-BASED CORRELATION

The foundational axiom of biological learning was postulated by Donald Hebb in 1949. Hebb proposed that synaptic efficiency is a function of the correlated activity between two neurons. Colloquially summarized as "Neurons that fire together, wire together," this rule implies that the brain learns by detecting statistical regularities in sensory input.

Mathematically, if neuron A consistently takes part in firing neuron B, the connection from A to B is strengthened. This mechanism allows the brain to perform unsupervised clustering, physically encoding associations between features that occur simultaneously in the environment (e.g., the smell of smoke and the sight of fire).

SPIKE-TIMING-DEPENDENT PLASTICITY (STDP)

Modern neuroscience has refined Hebb's macroscopic theory into a precise, millisecond-scale mechanism known as Spike-Timing-Dependent Plasticity (STDP). Unlike rate-based models, STDP operates on the precise timing of individual action potentials, introducing the critical element of causality.

The STDP rule adjusts the synaptic weight based on the relative timing difference (Δt) between the pre-synaptic input and the post-synaptic output:

- **Long-Term Potentiation (LTP)**: If the input spike arrives **before** the output spike ($\Delta t > 0$), it implies the input contributed to the firing. The synapse is strengthened to reinforce this causal link.
- **Long-Term Depression (LTD)**: If the input spike arrives **after** the output spike ($\Delta t < 0$), the input was irrelevant to the decision. The synapse is weakened.

This asymmetry allows the network to self-organize, naturally filtering out random noise while reinforcing specific spatiotemporal patterns.

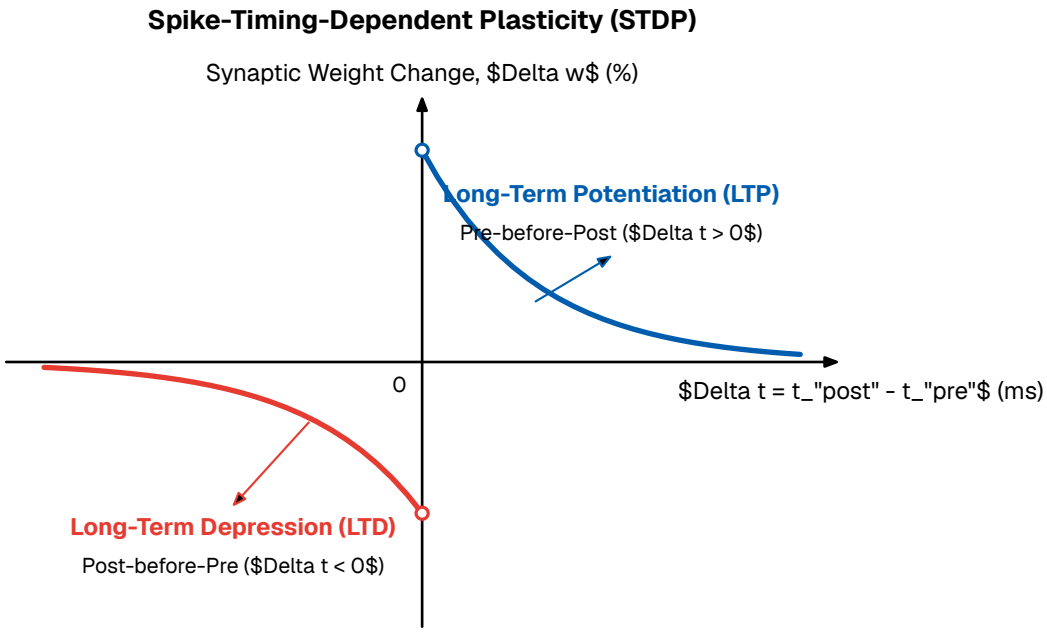


Figure 24: The STDP Learning Curve. Synaptic weight change is plotted against spike timing difference. Pre-before-post timing triggers strengthening (LTP), while post-before-pre triggers weakening (LTD).

HOMEOSTATIC PLASTICITY

If Hebbian mechanisms (LTP) were the sole drivers of plasticity, neural networks would be inherently unstable. A positive feedback loop would emerge where strengthened synapses drive higher firing rates, which in turn induce further strengthening, leading to runaway excitation (seizures). Conversely, unchecked LTD could silence a network entirely.

To maintain stability, the brain employs Homeostatic Plasticity (or Synaptic Scaling). This is a global regulatory mechanism that operates on a slower timescale (minutes to hours). It functions as a negative feedback loop: if a neuron's average firing rate exceeds a target set-point, the cell chemically downscales the strength of **all** its incoming synapses. This ensures that neurons remain within a sensitive dynamic range, preventing saturation regardless of how strong the inputs become.

THREE-FACTOR LEARNING: THE PATH TO REINFORCEMENT

While STDP is excellent for learning correlations, it is agnostic to the **value** of the outcome. A circuit might learn to associate "fire" with "touching," but STDP alone cannot encode whether this is good or bad. To learn tasks (Reinforcement Learning), the brain utilizes Neuromodulation.

In this "Three-Factor Rule," the synaptic update depends on:

- **The Pre-synaptic activity** (The input).
- **The Post-synaptic activity** (The action).
- **A Global Reward Signal**: The presence of neuromodulators like Dopamine.

Even if the timing is perfect for STDP, the synapse is only permanently strengthened if a neuromodulator arrives shortly after, signaling a “reward” or “novelty.” This bridges the gap between local synaptic events and global behavioral goals.

2.3 • TECHNICAL DETAILS OF MACHINE LEARNING

This chapter delineates the technical foundations of modern artificial intelligence, contrasting the established paradigms of Deep Learning (DL) with the emerging principles of Neuromorphic Engineering. We begin by analyzing the algorithmic architecture of standard Deep Learning, identifying the computational bottlenecks and energy inefficiencies inherent in its reliance on dense matrix multiplication and backpropagation. Subsequently, we introduce the technical framework of neuromorphic engineering, demonstrating how the translation of the biological principles discussed in the previous section—such as sparsity, event-driven processing, and local learning—can yield systems that vastly outperform conventional models in terms of energy efficiency and latency.

A critical distinction must be drawn between biological plausibility and bio-inspired engineering. From an engineering perspective, the primary objective is functional utility; a mechanism need not faithfully replicate biological reality to be valuable. It is important to recognize that evolution is a "satisficing" process—selecting for traits that ensure survival rather than finding mathematically optimal solutions. Therefore, an engineer may treat the brain merely as a source of heuristic inspiration rather than a blueprint to be copied dogmatically. However, the pursuit of biologically plausible systems remains vital. Not only does it offer potential distinct advantages in robustness and adaptability, but the creation of synthetic systems that respect biological constraints serves as a powerful verification tool for neuroscience, bridging the gap between artificial construction and biological understanding.

2.3.1 • OPTIMIZATION

To rigorously analyze intelligent systems, we must first abstract the learning process from its biological roots into a formal mathematical optimization problem. In this framework, "learning" is not treated as an emergent property of organic tissue, but as a search problem within a high-dimensional vector space.

Mathematical optimazation is the selection of a "best candiate" with regard to some defined criteria. A simple optimasation problem is finding the minimum of a one-dimentional function $f : \mathbb{R} \to \mathbb{R}$ here the "best canditade" is the lowest real number in the domain of the function. The criteria is "the lowest posible number $x \in \mathbb{R}$". If the function is convex it has a and smooth it has a well defiend "global minimum" and the function can be differentiated to find the global minimum. Any quantity that can be mesasured by a criteria can be optimized even functions themselves can be optimised.

Fundamentally, an artificial intelligence model operates as a function approximator. We assume the existence of an unknown underlying function $f : X \to Y$ that perfectly maps inputs to their target outputs, for example such a function could be one that perfectly describes the earth's wether pattern taking into account known

(and unkown) physics as well as all the particles with their mass and velocity in the atmosphere. This function may be fully deterministic or stocastic, in any case such a function is yet to be discoverd and the best we can do is to take guess at a much simpler function that still makes usefull predictions even if they are wrong to some degree. By adding tunable parameters to this function we can construct a family of hypothesis functions $f_{\boldsymbol{\theta}}(\boldsymbol{x})$ Here, $\boldsymbol{\theta} \in \mathbb{R}^d$ represents the state of the system—a vector containing all tunable parameters, such as synaptic weights, biases, or time constants. The dimensionality d of this space corresponds to the degrees of freedom of the model, and the precise configuration of $\boldsymbol{\theta}$ determines the system's behavior.

To guide the search for the optimal parameters $\hat{\boldsymbol{\theta}}$, one must quantify the divergence between the model's predictions and the ground truth. We define a scalar Loss Function $\mathcal{L}(\hat{\boldsymbol{y}}, \boldsymbol{y})$ that evaluates the error on a single data point, such as the Squared Error for regression or Cross-Entropy for classification. However, optimizing for a single example is insufficient for generalization. Instead, we seek to minimize the Empirical Risk (or Cost Function) $J(\boldsymbol{\theta})$, defined as the average loss over a dataset of size N:

$$J(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}\Big(f(\boldsymbol{x}_i, \boldsymbol{\theta}), \boldsymbol{y}_i \Big)$$

Consequently, the learning task is reduced to finding the global minimizer of this cost function:

$$\hat{\boldsymbol{\theta}} = \operatorname{argmin}_{\boldsymbol{\theta}} \; J(\boldsymbol{\theta})$$

Geometrically, the cost function $J(\boldsymbol{\theta})$ induces a complex structure known as the Optimization Landscape or Error Surface. In modern non-linear systems, this landscape is rarely convex; it is characterized by a multitude of local minima, saddle points, and plateaus. Navigating this non-Euclidean topology to find a low-energy state is the central challenge of AI engineering.

Since closed-form solutions for $\hat{\boldsymbol{\theta}}$ are generally intractable for complex non-linear functions, we rely on iterative optimization algorithms, principally Gradient Descent. This method relies on the mathematical principle that the gradient vector $\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$—the vector of partial derivatives with respect to all parameters—points in the direction of the steepest ascent of the function. To minimize error, the system must update its state in the direction opposite to the gradient. The update rule for a generic iteration t is given by:

$$\boldsymbol{\theta}_{t+1} \leftarrow \boldsymbol{\theta}_t - \eta \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_t)$$

In this equation, η (eta) represents the Learning Rate, a hyperparameter that governs the step size of the update. This formulation encapsulates the fundamental loop of modern AI: evaluate the current state, compute the gradient of the error, and incrementally adjust the parameters to descend the error surface toward convergence.

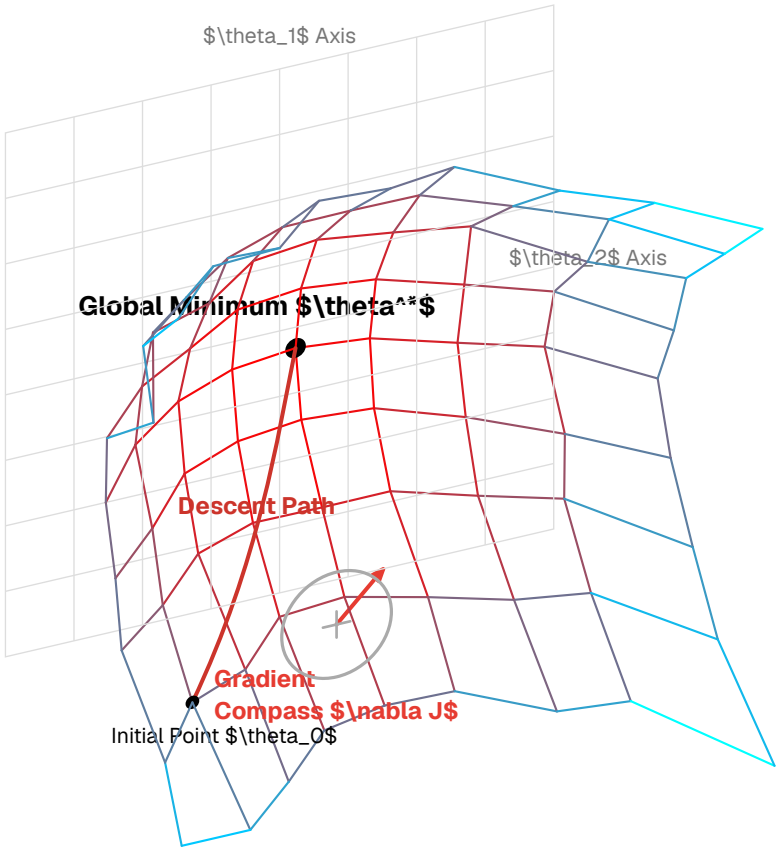


Figure 25: The Optimization Landscape. The system seeks to traverse the high-dimensional surface defined by $J(\boldsymbol{\theta})$ to find the global minimum $\boldsymbol{\theta}^*$, using the gradient ∇J as a navigational compass.

Altoguh gradient decent works remarkably well for its simplicity it has limitations. As already mentioned the method can get stuck in local minium. Furthermore the speed and effeciency may at times be slow. especcially for loss landscaped that appear flat. Perhaps the most crucial limitation is that the loss function must be differentialble, otherwise we cannot obtain a gradient. This becomes very important for optimasation of neuromorhpic systems.

UNSUPERVISED OPTIMIZATION

The optimization framework defined previously describes Supervised Learning, where the system is guided by explicit target labels $\boldsymbol{y}$. However, relying solely on labeled data is biologically implausible and engineering-wise inefficient. The vast majority of sensory data received by an intelligent agent is unlabeled; the retina receives photons, not pixel-wise classifications.

To handle this, we employ Unsupervised Learning. In this regime, the dataset consists only of input vectors $X = \{\boldsymbol{x}_1, ..., \boldsymbol{x}_N\}$. The optimization objective shifts from minimizing prediction error to minimizing representation error or energy.

Mathematically, the goal is often to learn the underlying probability distribution $P(\boldsymbol{x})$ of the data or to discover a lower-dimensional manifold $\mathcal{Z}$ that efficiently captures the structure of $\mathcal{X}$. A common formulation is the minimization of Recon-

struction Loss (as seen in Autoencoders), where the system attempts to compress the input into a latent code $\boldsymbol{z}$ and then reconstruct it:

$$J(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^{N} \| \boldsymbol{x}_i - f_{\text{decode}\left(f_{\text{encode}(\boldsymbol{x}_i;\boldsymbol{\theta})}\right)} \|^2$$

Alternatively, in energy-based models (which closely resemble physical thermodynamic systems), the optimization seeks to find a configuration of parameters that minimizes the "energy" of plausible data configurations while maximizing the energy of implausible ones.

This distinction is critical for Neuromorphic Engineering. Biological plasticity rules, such as those discussed in the following sections, are predominantly unsupervised. They function by detecting statistical correlations in the input stream (e.g., "edges generally occur in continuous lines") to build internal representations of the world without external supervision.

STOCHASTIC VS. BATCH OPTIMIZATION

The formulation above describes Batch Gradient Descent, where the gradient is computed over the entire dataset N before a single update is made. For modern high-dimensional datasets, this is computationally prohibitive. Furthermore, biological learning does not wait to experience "all of life" before adapting; it learns online.

To address this, modern AI employs Stochastic Gradient Descent (SGD) or Mini-Batch Gradient Descent. Instead of computing the exact gradient over N, the gradient is approximated using a small random subset (a batch) of data $B \ll N$:

$$\boldsymbol{\theta}_{t+1} \leftarrow \boldsymbol{\theta}_t - \eta \frac{1}{|B|} \sum_{(x,y) \in B} \nabla_{\boldsymbol{\theta}} \mathcal{L}(f(\boldsymbol{x}), \boldsymbol{y})$$

This introduces noise into the optimization trajectory. Paradoxically, this noise is beneficial: it prevents the system from getting stuck in shallow local minima and saddle points, allowing the optimization process to "jitter" its way toward more robust solutions. This mimics the noisy, event-driven updates seen in biological synaptic plasticity.

GENERALIZATION AND THE BIAS-VARIANCE TRADEOFF

Strictly minimizing the empirical cost $J(\boldsymbol{\theta})$ carries a risk. If the model capacity (d) is too large relative to the data size (N), the system may simply "memorize" the training examples, including their noise, rather than learning the underlying function f. To understand this failure mode, we decompose the generalization error into three components, known as the Bias-Variance Decomposition:

$$\text{Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$$

- **Bias (Underfitting)**: The error introduced by approximating a real-world problem with a simplified model (e.g., trying to fit a curve with a straight line). High bias causes the model to miss relevant relations between features and target outputs.
- **Variance (Overfitting)**: The error introduced by the model's sensitivity to small fluctuations in the training set. A high-variance model captures the random noise in the training data rather than the intended outputs.

This creates a fundamental tension: increasing model complexity decreases bias but increases variance. The objective of optimization is not zero training error, but finding the "sweet spot" in this tradeoff where the total generalization error is minimized.

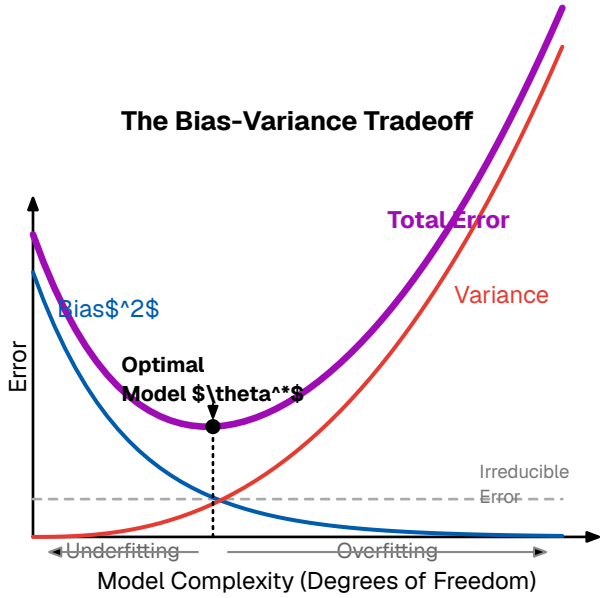


Figure 26: The Bias-Variance Tradeoff. As model complexity increases, bias (underfitting) decreases while variance (overfitting) increases. The optimal model exists at the trough of the total error curve.

To manage this tradeoff, optimization often includes Regularization terms (such as L_1 or L_2 penalties) that artificially constrain the complexity of the hypothesis space. In biological systems, this regularization is naturally enforced by metabolic constraints—the brain aggressively prunes weak connections to maintain a sparse, energy-efficient topology, effectively reducing variance by limiting the hardware available to overfit noise.

2.3.2 • DEEP LEARNING CONSTITUENTS

In the context of the optimization framework established previously, Artificial Neural Networks (ANNs) constitute a specific class of hypothesis functions formed by the hierarchical composition of simple, non-linear modules.

While the "Perceptron" introduced in the history chapter served as the atomic unit, modern Deep Learning aggregates these units into high-dimensional layers.

Mathematically, a Deep Neural Network with L layers is expressed not as a singular equation, but as a composite function $f_{\boldsymbol{\theta}}(\boldsymbol{x})$ mapping an input $\boldsymbol{x}$ to an output $\boldsymbol{y}$ through a chain of nested operations:

$$\boldsymbol{y} = f_L(...f_2(f_1(\boldsymbol{x})))$$

This depth is not arbitrary; it allows the system to learn hierarchical representations. The initial layers might detect simple edges or frequencies, while deeper layers recombine these primitives to recognize complex semantic concepts like "faces" or "syntax."

THE FORWARD PASS: AFFINE TRANSFORMATIONS

The computation of these layers during inference is known as the Forward Pass. For a standard Multi-Layer Perceptron (MLP), each layer l performs two distinct mathematical operations:

1. Affine Transformation: The input vector $\boldsymbol{a}^{(l-1)}$ is multiplied by a weight matrix $\boldsymbol{W}^{(l)}$ and shifted by a bias vector $\boldsymbol{b}^{(l)}$. This operation is linear and represents a rotation and scaling of the data manifold.
2. Non-Linear Activation: The result $\boldsymbol{z}^{(l)}$ is passed through a non-linear function $\sigma(\cdot)$.

$$\boldsymbol{z}^{(l)} = \boldsymbol{W}^{(l)}\boldsymbol{a}^{(l-1)} + \boldsymbol{b}^{(l)}$$

$$\boldsymbol{a}^{(l)} = \sigma\big(\boldsymbol{z}^{(l)}\big)$$

This seemingly simple structure contains the "Universal Approximation" capability of neural networks. However, the power of the network is entirely dependent on the choice of the non-linearity $\sigma(\cdot)$.

ACTIVATION DYNAMICS AND GRADIENT FLOW

If a network consisted only of linear transformations (just the $\boldsymbol{W}x + \boldsymbol{b}$ part), the entire deep stack would mathematically collapse into a single linear matrix multiplication. The activation function $\sigma(\cdot)$ introduces the necessary non-linearity to model complex real-world data.

However, the choice of $\sigma(\cdot)$ dictates not just expressivity, but trainability. During optimization, the gradient must flow backwards through these functions.

Sigmoid / Tanh (The Old Paradigm): Historically, functions like the Sigmoid were used because they mimic the firing rate of a biological neuron (saturating at 0 and 1). However, their derivatives are strictly less than 1 (max 0.25). As gradients propagate back through many layers, these small numbers multiply, causing the signal to decay exponentially to zero. This is the Vanishing Gradient Problem. ReLU (The Modern Standard): To enable deep learning, modern networks employ the Rectified Linear Unit, $f(x) = \max(0, x)$. Its derivative is either 0 or 1. This "gating" property preserves

the magnitude of the gradient, allowing error signals to travel through deep structures without vanishing.

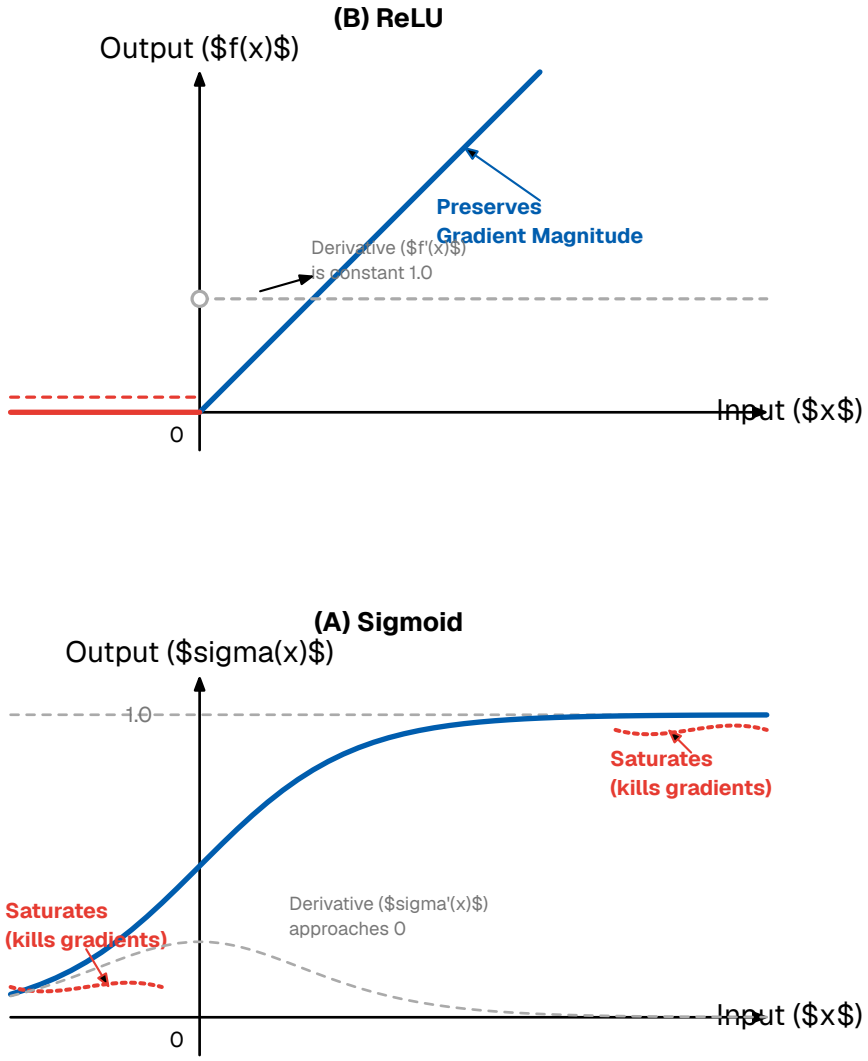


Figure 27: Activation Functions. The Sigmoid (left) saturates, killing gradients. The ReLU (right) preserves gradient magnitude for positive inputs.

THE BACKWARD PASS: COMPUTATIONAL GRAPHS

To update the parameters $\boldsymbol{W}$ and $\boldsymbol{b}$, the system must attribute the total error $J(\boldsymbol{\theta})$ to specific weights. This is achieved via Backpropagation.

Conceptually, Backpropagation is simply the recursive application of the Chain Rule of calculus. If we view the network as a computational graph, the gradient for a weight w in layer l is computed by propagating the error δ from the layer above:

$$\frac{\partial \mathcal{L}}{\partial w} = \underbrace{\frac{\partial \mathcal{L}}{\partial a}}_{\text{Error from above}} \cdot \underbrace{\frac{\partial a}{\partial z}}_{\text{Activation derivative}} \cdot \underbrace{\frac{\partial z}{\partial w}}_{\text{Input value}}$$

In modern frameworks (like PyTorch or TensorFlow), this is implemented via Automatic Differentiation (AutoDiff). The system builds a dynamic graph of operations during the forward pass and traverses it in reverse to compute exact gradients.

DEEP LEARINING COMPUTATIONS: DENSE MATRIX OPERATIONS

While the equations above describe the behavior of individual neurons, modern Deep Learning does not compute them one by one. To achieve the throughput required for training, these operations are vectorized. The affine transformation for an entire layer is executed as a Dense Matrix Multiplication (GEMM):

$$\boldsymbol{Z} = \boldsymbol{W} \times \boldsymbol{A} + \boldsymbol{b}$$

Where $\boldsymbol{W}$ is a matrix of dimensions $N_{\text{out}} \times N_{\text{in}}$. Crucially, the backward pass (gradient computation) also reduces to matrix multiplication, typically involving the transpose of the weight matrix $\boldsymbol{W}^T$.

This mathematical reality is the defining characteristic of modern AI hardware usage. A deep network is effectively a sequence of massive matrix multiplications. While this structure allows for efficient parallelization on SIMD (Single Instruction, Multiple Data) hardware like GPUs, it creates a massive dependency on memory bandwidth. The entire weight matrix $\boldsymbol{W}$—which can reach gigabytes in size for modern Transformers—must be loaded into the processor registers for every single inference step.

This structural reliance on moving massive dense matrices provides the context for the primary bottleneck of modern AI, discussed in the following section.

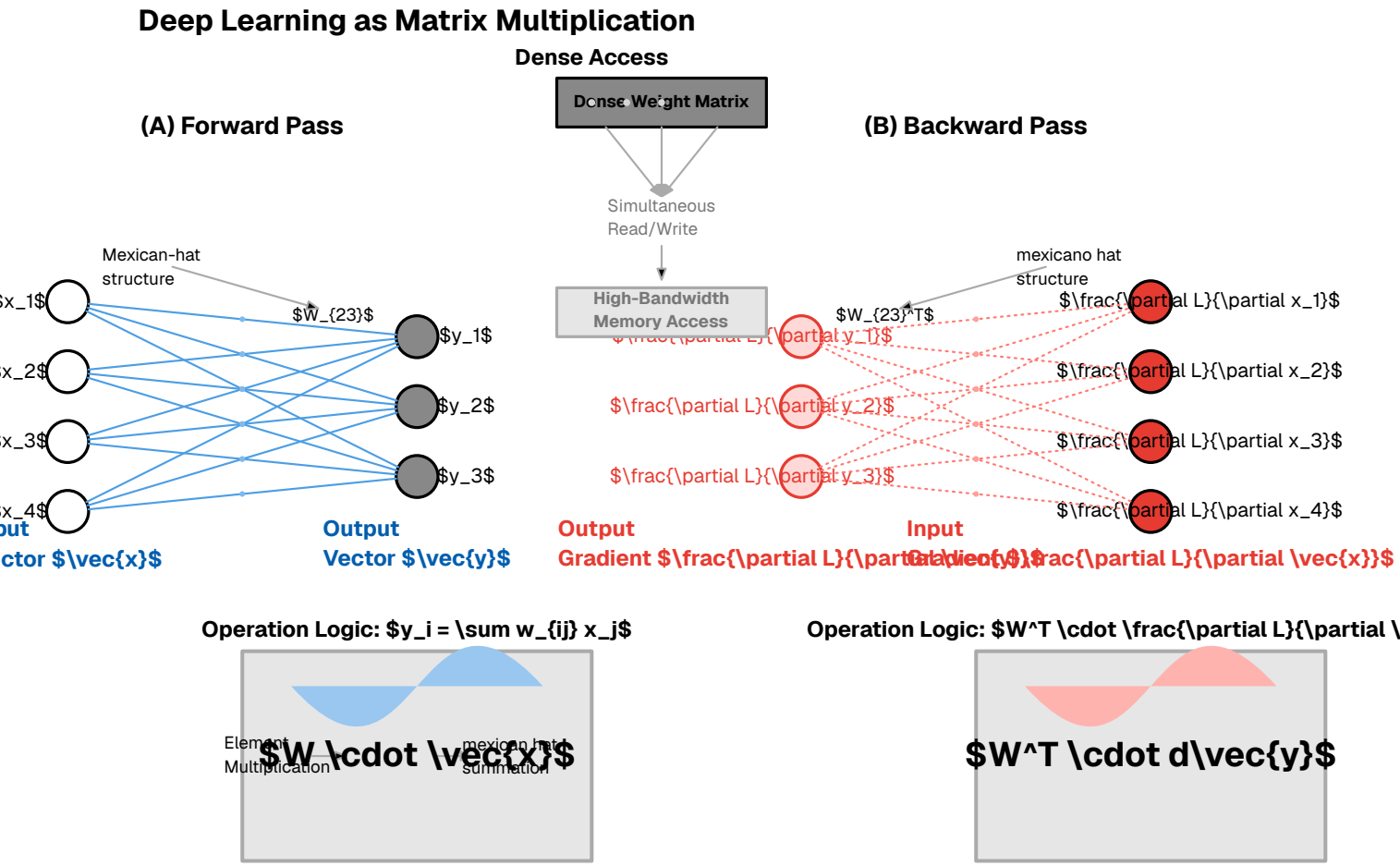


Figure 28: Deep Learning as Matrix Multiplication. Both forward and backward passes rely on dense matrix-vector products, necessitating high-bandwidth memory access.

2.3.3 • WHY IS DEEP LEARNING INEFFICIENT?

While the matrix-centric formulation of Deep Learning enables high-throughput parallelization on GPUs, it fundamentally conflicts with the physical constraints of modern computing hardware. As models scale to billions of parameters, the primary bottleneck shifts from algorithmic capability to physical realizability. This inefficiency manifests in three distinct engineering dimensions: data movement, dense computation, and global synchronization.

THE VON NEUMANN BOTTLENECK & DATA MOVEMENT

The most significant physical limitation is the Von Neumann Architecture, which physically separates the Processing Unit from the Memory Unit. For Deep Learning, this is catastrophic. A neural network is defined by its weight matrix $\boldsymbol{W}$. To perform a single inference step, the processor must fetch the entire weight matrix from off-chip DRAM to on-chip registers, perform the calculation, and write results back.

According to data from Horowitz and Dally [1], fetching a 32-bit value from off-chip DRAM consumes approximately 640 pJ, whereas performing the floating-point

addition on that value consumes only 0.1 pJ. The system burns 99.9% of its energy moving data to the calculator, and only 0.1% actually doing the calculation.

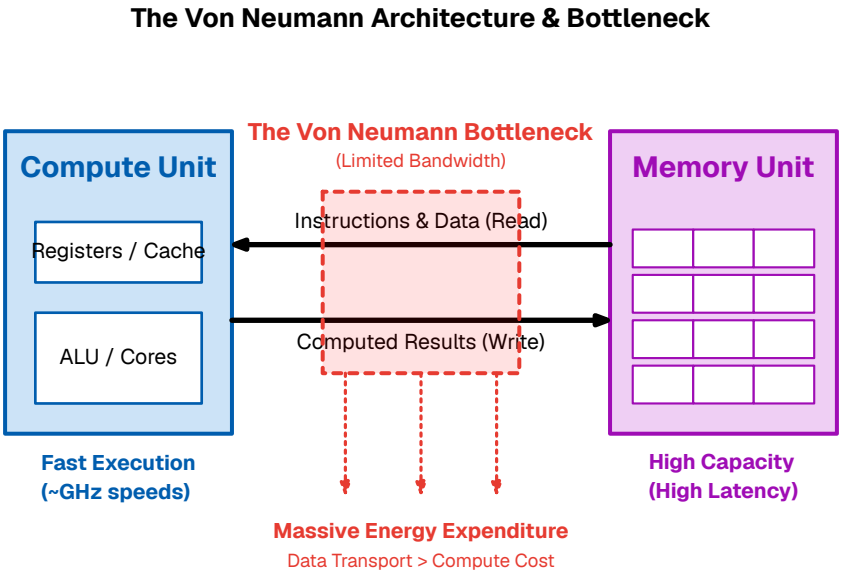


Figure 29: The Von Neumann Bottleneck. The separation of memory and compute forces massive energy expenditure on data transport.

DENSE PROCESSING OF SPARSE DATA

Standard Deep Learning implementations rely on Dense Matrix Multiplication (GEMM). This approach is algorithmically rigid: it executes the same number of operations regardless of the data content.

Real-world data is often sparse (containing many zeros), and the ReLU activation function naturally produces activation maps where 50-80% of values are zero. However, a standard GPU is "blind" to this sparsity. It will dutifully fetch a zero from memory, load it into a register, and multiply it by a weight ($0 \times w = 0$), consuming energy and cycles to produce a null result.

In a biological or neuromorphic system, a "zero" is simply the absence of an event. Nothing is transmitted, and absolutely no energy is consumed. Deep Learning's inability to exploit this silence represents a massive structural inefficiency.

THE HIGH COST OF SYNCHRONY

Deep Learning hardware is typically Synchronous, meaning every component operates in lockstep with a global clock. This introduces a severe overhead known as the Clock Distribution Penalty.

To ensure that billions of transistors switch at the exact same moment, the chip must drive a high-frequency clock signal across the entire silicon die. Charging and discharging the capacitive wires of this clock tree occurs billions of times per second, regardless of whether the chip is doing useful work. In many high-performance

processors, the clock network alone can consume 30% to 40% of the total power budget.

Furthermore, this global synchronization enforces a "worst-case" latency. If one part of the matrix multiplication finishes early, it must sit idle and wait for the slowest part to finish before the next clock cycle can begin.

BACKPROPAGATION AND GLOBAL DEPENDENCIES

Finally, the learning algorithm itself—Backpropagation—imposes severe constraints on memory and latency. Backpropagation is non-local in both time and space.

To update a specific weight w_{ij} in the first layer of a deep network, the system cannot act immediately. It must:

1. Wait for the Forward Pass: The input must propagate through the entire network to the output layer to generate a prediction.
2. Calculate Global Error: The system computes the loss $J(\boldsymbol{\theta})$ based on the global output.
3. Wait for the Backward Pass: The error gradient must be propagated all the way back from the output to the input.

This creates a Locking Problem. The activations of every intermediate layer must be stored in high-speed memory (VRAM) for the duration of the entire pass, preventing that memory from being reused. This memory footprint grows linearly with network depth, often limiting the size of models that can be trained. Additionally, the weight update is dependent on the global state of the network, meaning a local synapse cannot adapt to local changes instantly; it is shackled to the global error loop.

2.3.4 • PRINCIPLES OF NEUROMORPHIC ENGINEERING

To address the fundamental inefficiencies of the Von Neumann architecture, we turn to the paradigm of Neuromorphic Engineering. The term, coined by physicist Carver Mead in the late 1980s, is derived from the Greek roots *neuro* (relating to nerves or the nervous system) and *morphe* (meaning form or shape).

Literally translating to "taking the form of the brain," the term was not intended to describe software simulations of neural networks. Rather, it described a specific hardware design philosophy: the construction of electronic circuits that utilize the inherent physics of silicon to mimic the biophysics of neural tissue.

Mead's foundational insight was that the physical equations governing the flow of ions through biological channels (Boltzmann statistics) are mathematically identical to the equations governing the flow of electrons through a transistor operating in its "subthreshold" (weak inversion) region.

$$I_{\text{channel}} \propto e^{\kappa \frac{V_g}{U_T}} \text{ (Transistor)} \longleftrightarrow I_{\text{membrane}} \propto e^{\frac{V_m}{V_T}} \text{ (Neuron)}$$

Therefore, "Neuromorphic" does not simply mean "AI inspired by the brain." It specifically refers to systems that replicate the physical topology and analog dynamics of the nervous system to achieve computation. While modern interpretations have expanded to include digital implementations (like Intel's Loihi), the core definition remains rooted in mimicking the brain's structural "form"—specifically its parallelism, connectivity, and local processing—rather than just its mathematical outputs.

The inefficiencies described above—the memory wall, the energy cost of dense processing, and the overhead of global synchronization—are not fundamental limits of computation. Rather, they are artifacts of the Von Neumann architecture. As established in the *History & Developments* section, physicist Carver Mead identified this divergence as early as 1990, proposing that to achieve the efficiency of the brain, we must adopt the physics of the brain.

Neuromorphic Engineering is the translation of the Biological Principles discussed in the previous chapter into silicon hardware. It replaces the rigid, clock-driven logic of standard computing with the adaptive, event-driven dynamics of neural tissue. This approach rests on three architectural pillars that directly address the bottlenecks of Deep Learning:

CO-LOCATION OF MEMORY AND COMPUTE (THE SYNAPTIC PRINCIPLE)

To dismantle the Von Neumann bottleneck, neuromorphic architectures eliminate the separation between the processor and the memory. In the biological brain, there is no separate "RAM" module; memory is stored in the synaptic weights themselves, right where the processing (integration of current) occurs.

Neuromorphic chips replicate this by distributing memory across the silicon die. Each artificial neuron possesses its own local memory to store its state and synaptic weights. By processing data **in situ**, the massive energy cost of shuttling weights back and forth is eliminated. This is the engineering realization of the Synaptic Plasticity mechanisms discussed earlier: computation and storage are physically inseparable.

EVENT-DRIVEN ASYNCHRONY (THE ACTION POTENTIAL PRINCIPLE)

To address the "Dense Processing of Sparse Data" and the "Clock Distribution Penalty," neuromorphic systems abandon the global clock. Instead, they operate asynchronously, driven strictly by the arrival of data.

This mimics the All-or-None Law of the biological neuron. Just as a neuron remains quiescent until its membrane potential reaches a threshold, a neuromorphic circuit consumes negligible power until an event (a spike) arrives. If a part of the network is not currently processing information, it effectively shuts down. This "activity-gating" ensures that power consumption scales linearly with the complexity of the task, rather than the size of the network—a critical advantage for processing sparse real-world sensory data.

SPARSE COMMUNICATION (THE SPIKE PRINCIPLE)

Finally, to solve the bandwidth issue, neuromorphic systems utilize Spikes for communication. Unlike the 32-bit floating-point numbers used in Deep Learning, a spike is a binary event that carries no payload other than its source address and its timing.

This corresponds to the biological principle of Temporal Coding. Information is not encoded in the complex magnitude of a signal, but in the precise timing of events (Inter-Spike Intervals or Time-to-First-Spike). This allows the system to compress high-dimensional information into sparse, energy-efficient pulse trains, drastically reducing the bandwidth required to transmit information between neurons.

CONCLUSION OF THE FRAMEWORK

By inverting the Von Neumann paradigm, Neuromorphic Engineering offers a path to artificial intelligence that operates within the energy envelope of biological systems. The following sections detail the specific algorithmic implementation of these principles: the Spiking Neural Network (SNN).

Neuromorhic engineering and neuromorphic computing is any system that mimics or takes advantage of key mechanisms of the nervous system. The great promise of neuromorphic is to act as a successor to deep learing and take on tasks which needs greater inteligne ability to adapt without breaking the energy and resource budget. As mentioned Carver Mead pionered this feild and coined the term. But neuroscince has gone a long way since and we can draw more inpiration.

The unifying mechanism that all neuromorhic systems use is the spike or event just like the nervous system coverd in Section 2.2. A neuromorphic camera can use the change in luminosity of a pixel to send out an event, this way only changes in the scene are recorded and the camera becomes super effecient.

Neuromorphic computers can also use the fact that summation of currents in the neuron is analog and can be computed very effeciently using analog summing circuits.

Neuromorhic ideology is in creating algrithms directly embedded in hardware

2.3.5 • LEARNING IN NEUROMORPHIC SYSTEMS

In the optimazation and deep learing section we saw that training deep neural networks can be achevied with gradient decent this starts to break down when we try to apply it to neuromor

loss function needs to influence the synaptic weigts. weights does not know anything about a global signal (execpt for feedback dopamine) weights have to

Consequently, standard backpropagation cannot be directly applied to SNNs. Gradients calculated using the chain rule become zero or undefined at the spiking neurons, preventing error signals from flowing backward through the network to

update the weights effectively. This incompatibility represents a substantial obstacle, as it seemingly precludes the use of the highly successful and well-understood gradient-based optimization toolkit that underpins much of modern AI.

To maintain this critical regime, modern neuromorphic chips implement homeostatic plasticity—algorithms that automatically scale synaptic weights to keep the mean firing rate within a target range, ensuring signals can propagate through deep layers without fading out or exploding.

Surrogate Gradients: A popular approach involves using a "surrogate" function during the backward pass of training. While the forward pass uses the discontinuous spike generation, the backward pass replaces the step function's derivative with a smooth, differentiable approximation (e.g., a fast sigmoid or a clipped linear function). This allows backpropagation-like algorithms (often termed "spatio-temporal backpropagation" or similar) to estimate gradients and train deep SNNs, albeit with approximations.

2.3.6 • NEUROMORPHIC HARDWARE TECHNIQUES

In contrast to deep learing hardware. neuromorphic hardware is not constrained to the von neuman architechture and thus the bottleneck adress event representation make reference to macro-motif section

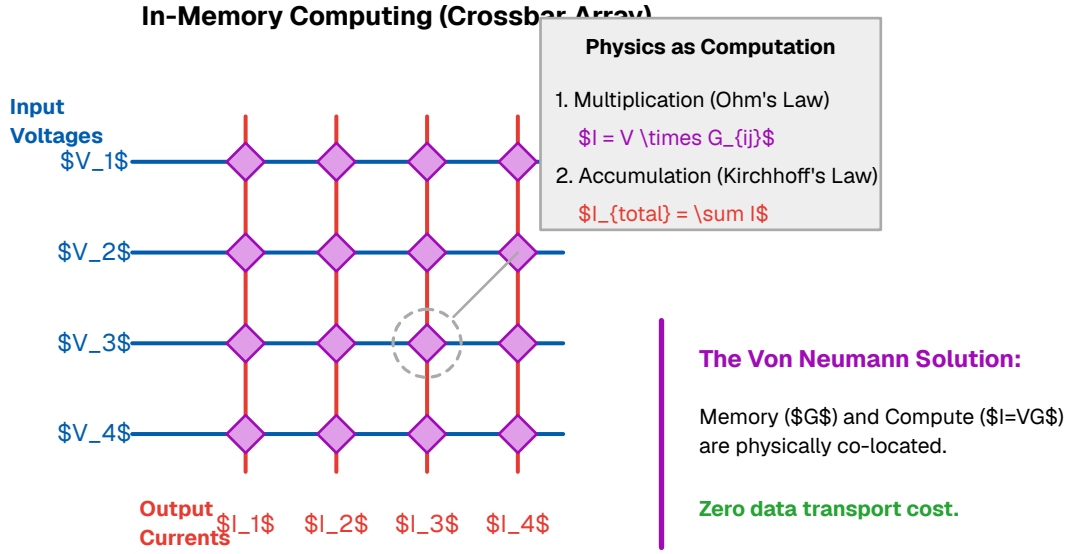


Figure 30: In-Memory Computing via a Crossbar Array. Unlike von Neumann architectures, memory and computation are physically co-located. Input voltages (V, representing activations) are applied to the wordlines. The memory elements at the junctions hold programmable conductances (G, representing synaptic weights). Multiplication is naturally performed at each junction by Ohm's Law (I=V×G), and the resulting currents are summed along the bitlines via Kirchhoff's Current Law (ΣI). This allows dense matrix-vector multiplications to occur in a single analog time step with zero data transport cost.

crossbar array looks like a neural network!

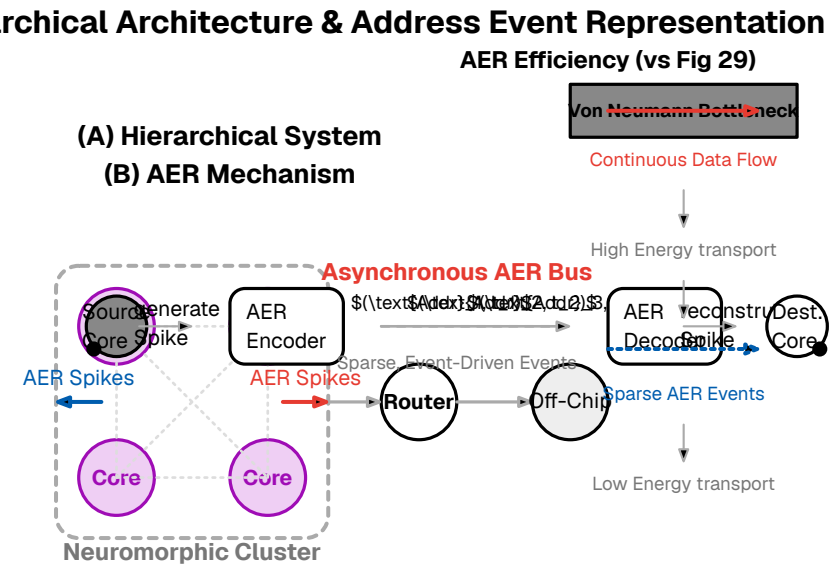


Figure 31: Architectural Comparison. (Left) The Von Neumann architecture separates memory and compute, creating a bottleneck. (Right) The Neuromorphic architecture co-locates them, mimicking the distributed topology of biological neural networks.

2.4 • NEUROMORPHIC STATE OF THE ART

Disentangling core computational mechanisms from biological implementation details is a major ongoing challenge in neuroscience and neuromorphic engineering. Some complex molecular processes might be essential for learning or adaptation, while others might primarily serve metabolic or structural roles not directly involved in the instantaneous computation being modeled. The principles of neuromorphic computing, born from Carver Mead's vision and informed by modern neuroscience, have matured from theoretical concepts into a vibrant field of applied research. This progress is best seen in two key areas: the development of specialized, brain-inspired hardware and the creation of sophisticated software frameworks for simulating and deploying spiking neural networks (SNNs).

2.4.1 • APPLIED NEURAL CODES

A central challenge in neuromorphic engineering is encoding information into spikes. The most "standard" method is Rate Coding (frequency of spikes = intensity), but this is slow and energy-inefficient. To solve this, Simon Thorpe proposed Rank Order Coding. Thorpe observed that the human visual system processes images far too quickly for neurons to average spikes over time. Instead, he proposed that information is encoded in the order of firing. The most strongly activated neurons fire first.

The "N-of-M" Strategy: To implement this efficiently in hardware, engineers often use an "N-of-M" coding scheme.

- M (Population): A large pool of potential neurons (e.g., 1000).
- N (Active): Only the first N neurons (e.g., 50) to spike are transmitted.
- Mechanism: Once N spikes are received, the system inhibits the rest. This guarantees extreme sparsity (low power) and filters out "noise" (late spikes).

This approach transforms time into a priority queue. A downstream neuron does not wait for a "frame" to finish; it begins computing as soon as the earliest, most salient spikes arrive.

2.4.2 • LEARING

The most biologically plausible learning algorithm is STDP. Unlike Deep Learning, which updates weights based on a global error calculated at the output, STDP updates weights based on local causality between two connected neurons.

Causal (LTP): If the input spike (pre) arrives before the output spike (post), the synapse is strengthened (Long-Term Potentiation). The input "caused" the firing.

Acausal (LTD): If the input spike arrives after the output spike, the synapse is weakened (Long-Term Depression). The input was irrelevant to the decision.

STDP allows networks to self-organize and detect repeating patterns in data without labeled supervision. However, on its own, it struggles to reach the high accuracy of modern supervised classifiers.

SURROGATE GRADIENTS

To achieve "State-of-the-Art" (SOTA) performance on complex tasks (like ImageNet or Speech Recognition), modern neuromorphic engineers often hybridize biology with Deep Learning.

The core problem is that spikes are non-differentiable (a step function has zero derivative everywhere), which breaks standard Backpropagation. The solution is Surrogate Gradient Learning.

Forward Pass: The hardware uses the true, crisp spiking physics (non-differentiable).

Backward Pass: The learning algorithm substitutes the spike with a smooth "surrogate" function (like a sigmoid) to calculate gradients.

This allows us to train SNNs using powerful optimizers (like Adam) and frameworks (like PyTorch), transferring the trained weights to the neuromorphic chip for efficient inference.

2.4.3 • COMPLEX DYNAMICAL MODELS

Neuromorphic is not just about making processors but also about building machines and models to better understand the biology

Beyond these standard algorithms, there is a rich landscape of more complex theoretical models. While we will not utilize them in this specific implementation, they represent the frontier of the field:

Reservoir Computing (Liquid State Machines): Using a chaotic, randomly connected "tank" of neurons to project inputs into high-dimensional space before a simple readout layer.

Equilibrium Propagation: A physics-based learning rule that relaxes a network to an energy minimum, avoiding the need for a separate backward pass.

Attractor Networks: Recurrent networks (Ring or Line attractors) that maintain stable states (memories) even in the absence of input, crucial for spatial navigation and working memory.

2.4.4 • NEUROMORPHIC SENSORS

2.1. The Paradigm Shift: From Frame-Based to Event-Based Sensing

Traditional sensory acquisition systems, particularly in computer vision and audio processing, have historically relied on the von Neumann architecture's separation of memory and processing, coupled with a clock-driven sampling approach. In this

conventional paradigm, sensors capture the state of the entire environment at fixed temporal intervals—generating frames or samples regardless of the scene's activity. While effective for static analysis, this method generates redundant data streams that impose significant latency, bandwidth, and power consumption penalties, particularly in high-speed or sparse-signal environments.

In contrast, neuromorphic engineering, a field pioneered by Carver Mead in the late 1980s, seeks to emulate the biological principles of the mammalian nervous system. Neuromorphic sensors abandon the global shutter and fixed clock in favor of asynchronous, event-driven acquisition. Information is encoded not as absolute values at a fixed rate, but as a sparse stream of "events" or "spikes" triggered only when a significant change in the physical stimulus occurs. This bio-inspired approach offers theoretical and practical advantages in terms of temporal resolution, dynamic range, and energy efficiency (pJ/event), effectively shifting the computational burden from raw data processing to sparse event management.

2.2. Neuromorphic Vision Sensors (DVS)

The most mature realization of this paradigm is the Dynamic Vision Sensor (DVS), often referred to as the event camera. Unlike standard Active Pixel Sensors (APS) that integrate photon counts over a fixed exposure time, each pixel in a DVS operates independently and asynchronously.

The fundamental operation of a DVS pixel involves continuous monitoring of the log-intensity of the incident light. An event e is generated at time t when the change in logarithmic intensity exceeds a preset threshold θ:

$$\Delta \ln(I) = \ln(I(t)) - \ln(I(t - \Delta t)) \geq \pm\theta$$

where I(t) is the photocurrent at time t. This mechanism yields a stream of events characterized by a tuple (x,y,t,p), representing the pixel coordinates, the microsecond-resolution timestamp, and the polarity of the brightness change (ON or OFF).

Recent literature highlights three key advantages of this architecture:

Temporal Resolution: Event cameras achieve effective frame rates equivalent to several kilohertz, with latencies in the microsecond range, making them ideal for high-speed robotics and ballistics tracking.

Dynamic Range (DR): Because individual pixels do not share a global exposure time, DVS pixels do not saturate easily. Modern neuromorphic sensors boast dynamic ranges exceeding 120 dB, compared to the ~60 dB typical of standard industrial cameras, allowing robust operation in scenes with simultaneous extreme brightness and darkness.

Data Sparsity: In static scenes, a DVS generates near-zero output, drastically reducing power consumption and downstream processing requirements compared to frame-based cameras that output constant data regardless of content.

2.3. Auditory and Tactile Modalities

While vision sensors dominate the field, the principles of neuromorphic sensing extend to other modalities, mirroring the specialized transduction mechanisms of the biological cochlea and skin.

2.3.1. Silicon Cochlea

Neuromorphic auditory sensors, or "silicon cochleas," emulate the basilar membrane's hydrodynamics. Instead of performing a global Fourier Transform (FFT) on a sampled audio waveform, these sensors utilize a cascade of analog band-pass filters. Each filter channel operates independently, generating spikes when the energy in its specific frequency band exceeds a threshold. This architecture provides high-fidelity temporal information crucial for tasks such as sound source localization and separation, often achieving lower latency and power consumption than DSP-based solutions. 2.3.2. Neuromorphic Tactile Sensing

Electronic skins (e-skins) and neuromorphic tactile sensors are an emerging frontier, designed to provide robots with high-frequency feedback for manipulation tasks. Recent advances utilize triboelectric, piezoelectric, or piezoresistive materials coupled with event-based readout circuits. These sensors encode pressure, vibration, and shear force changes as asynchronous spikes, allowing for the rapid detection of slip events—a critical capability for stable grasping that mimics the fast-adapting mechanoreceptors (e.g., Meissner corpuscles) in human skin.

2.4. Processing Events: Spiking Neural Networks (SNNs)

The asynchronous nature of neuromorphic sensors necessitates a departure from standard Convolutional Neural Networks (CNNs), which are optimized for dense, synchronous matrix multiplications. The natural processing counterpart for event streams is the Spiking Neural Network (SNN).

In an SNN, artificial neurons maintain an internal membrane potential that integrates incoming spikes over time; the neuron "fires" an output spike only when this potential crosses a threshold. This compatibility has driven the development of specialized neuromorphic hardware accelerators, such as Intel's Loihi and SynSense's Dynap-CNN, which support massive parallelism and local synaptic plasticity. These chips enable edge-native learning and inference with power budgets in the milliwatt range, significantly lower than standard GPU-accelerated embedded systems

2.4.5 • NEUROMORPHIC COMPUTERS

The primary goal of neuromorphic hardware is to escape the von Neumann bottleneck and emulate the power efficiency and massive parallelism of the brain. Two landmark systems define the state of the art:

IBM TrueNorth: A prominent early example, TrueNorth is a fully digital, real-time, event-driven chip. It consists of 4,096 "neurosynaptic cores," collectively housing one million digital neurons and 256 million synapses. Its architecture is explicitly non-von Neumann; processing and memory are tightly integrated within each core.

TrueNorth's key achievement is its staggering power efficiency: it can perform complex SNN inference tasks (like real-time video object detection) while consuming only tens of milliwatts—orders of magnitude less than a CPU or GPU performing a similar task. However, its architecture is largely fixed, making it a powerful "inference engine" but less flexible for researching novel, on-chip learning rules.

Intel Loihi (and Loihi 2): Intel's line of neuromorphic research chips, starting with Loihi in 2017, represents a significant step towards flexible, on-chip learning. Like TrueNorth, Loihi is an asynchronous, event-driven digital chip, but with a key difference: it features programmable "learning engines" within each of its 128 neuromorphic cores. This allows researchers to implement and test dynamic learning rules, such as STDP and its variants, directly on the hardware in real-time. The second generation, Loihi 2, further refines this with greater scalability, improved performance, and more advanced, programmable neuron models, positioning it as a leading platform for cutting-edge neuromorphic algorithm research.

Neuromorphic sensors ...

2.4.6 • SIMULATION AND SOFTWARE FRAMEWORKS

Before algorithms can be deployed on specialized hardware, they must be designed, tested, and validated. This is the role of SNN simulators, which function as the "TensorFlow" or "PyTorch" of the neuromorphic world.

Brian: A highly flexible and popular SNN simulator used extensively in the computational neuroscience community. Its strength lies in its intuitive syntax, which allows researchers to define neuron models and network rules directly as a set of mathematical equations (e.g., the differential equations of a Leaky Integrate-and-Fire neuron). This makes it an ideal tool for exploring the detailed dynamics of biologically realistic models.

Nengo: A powerful, high-level framework that functions as a "neural compiler." Nengo is built on a strong theoretical foundation (the Neural Engineering Framework) that allows users to define complex computations and dynamical systems in high-level Python code. Nengo then "compiles" this functional description into an equivalent SNN. Its key advantage is its backend-agnostic nature; the same Nengo-defined network can be run on a standard CPU, a GPU, or deployed directly to neuromorphic hardware like Intel's Loihi.

2.5 • RESEARCH GAPS

Despite this immense progress in hardware and software, a fundamental challenge remains, creating a critical research gap: the training problem.

Mainstream deep learning has a powerful, universal tool: backpropagation. Neuromorphic computing does not yet have a clear equivalent. While these systems exist, they still struggle with finding an efficient, scalable, and powerful learning algorithm that is both biologically plausible and computationally effective. This gap manifests in several ways:

- Limited Supervised Learning

"Local" rules like STDP are fundamentally unsupervised. They are excellent for finding patterns and correlations but struggle with complex, task-driven "supervised" problems (e.g., "classify this audio signal into one of ten specific words").

- The Conversion Compromise

A popular workaround is to first train a conventional, non-spiking ANN using backpropagation, and then "convert" its weights to an SNN for efficient inference. This method, while practical, is a compromise. It discards the rich temporal dynamics SNNs are capable of and does not represent true "neuromorphic learning."

- The Surrogate Gradient Challenge

The firing of a spiking neuron is a non-differentiable event, which makes it incompatible with standard backpropagation. New methods, like "surrogate gradient" learning, attempt to approximate this spike event with a smooth function to enable gradient-based learning, but this is an area of intense and ongoing research.

This thesis confronts this central challenge: How to effectively and efficiently train spiking neural networks for complex, real-world temporal tasks. While hardware like Loihi provides the platform for on-chip learning, it still requires a robust and scalable algorithm. The existing approaches of ANN-to-SNN conversion or simple Hebbian rules are insufficient.

This thesis proposes a method to bridge this gap by... (...for example: "...developing a novel, event-driven surrogate gradient algorithm capable of training deep SNNs directly in the temporal domain," or "...introducing a hybrid learning rule that combines the efficiency of STDP with the task-driven power of error-based feedback," or "...proposing a new architecture for temporal credit assignment that is both hardware-friendly and scalable.")

3 • METHOD

This chapter details implementations of neuromorphic methods to address the fundamental limitations of standard deep learning described in Section 2.2. The approach presented here aligns with the constraints of biological substrates: sparsity, asynchrony, and locality. We propose a neuromorphic architecture designed to maximize energy efficiency and computational robustness

3.1 • DATA

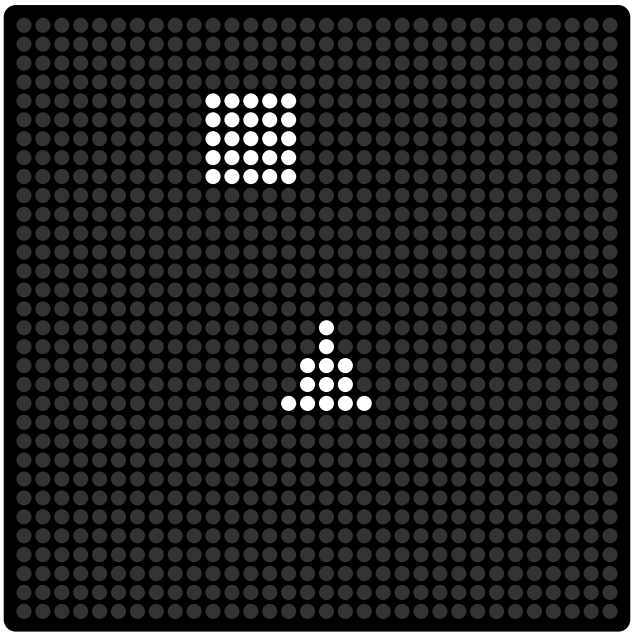


Figure 32: In-memory

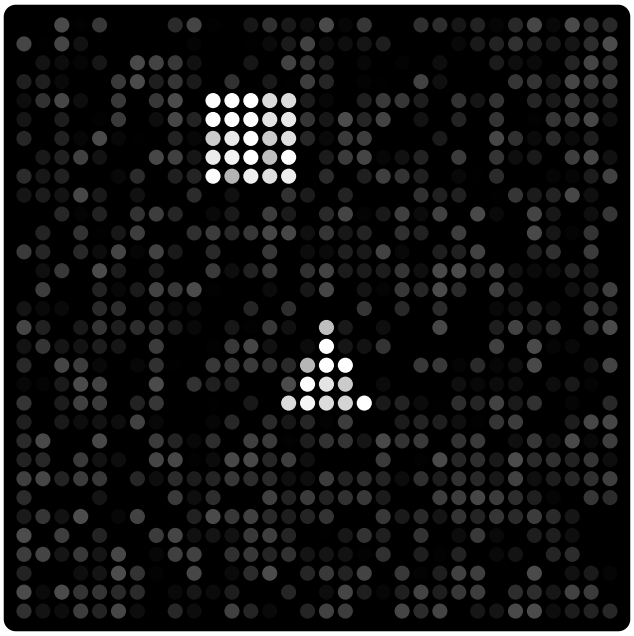


Figure 33: In-memory

3.2 • INFORMATION REPRESENTATION

The choise of a coding shceme puts key constraints on the design any processing system as it lays the founation for the flow of information. In Section 2.2.4 and Section 2.4.1 we dicussed several candidate neural codes that are both biologically plausible and have been used in previous neuromorphic systems. The neural code plays a large role in determening the effecieny of the system. Many neuromorphic systems use rate code as it is easy to translate values it is straight forward. any float value can be encoded as a rate code and integrate and fire neurons work well with this encoding. An IF neuron using rate code can reduce the multiply and add operations with just add operations. each spike arriving at a synapse adds its weight to the total. It is easy to see that for a rate code a higher rate input means more of that input contributing to the total sum. Using a rate code we can convert multiply and accumulate operations to just a series of accumulate operations by discretetizing the input values. The main reason for why this thesis will not use a rate code is that it is relativly ineficient and slow compared to a temporal code. In a temporal code only one spike is required to establish its value in relation to other synapses, making a single event much more dense in information. This is especially important for systems that do not have the connection density of a biological brain and must use shared buses that can be congested if too many spikes are present on the bus. Secondly to determine the value of a rate code you have to take an average of multiple spikes, imposing a delay. Using a TTFS encoding requires more sophisticated neuron models than a simple integrate and fire, it need more bookkeeping to keep track of the relative arival of incoming spikes and as mentioned in Section 2.2.4 and as we will explain by using a temporal encoding we need to handle the phase abmiguity problem. A neural code should have the following characteristics:

- Fast
- Effecient in terms of neurons used
- Able to encode a wide range of stimuli
- Robust to noise

Visual tasks which is what the application this thesis focuses on

1. a population code is used in conjunction with a temporal code
2. Population code is used alone (eg individual order within a population does not matter)

That beeing said and as discussed in Section 2.2.4 combining neural codes and using the right one for the job is also an important aspect to consider, there is not a clear consenus on which code is better it is a matter of the task at hand. As we also have talked about, Combining differnt codes toghether can make for a powerful representation of information. This thesis focuses on visaul recognition and a TTFS

combined with population code is the representation of choise. The popultion code will represent a distinct pattern in an image. This can be set up to detect lines and primitve shapes, deeper layers will use populations of primitve shapes to detect more complex features like circles and boxes and so on.

3.2.1 • ENCODING

To convert a float value (like a pixel value from an image to a TTFS encoding we first need to find the range of the pixels from minimum to maximum value. Then we need to decide on what featere of interest we wish to represent, a natural choise is pixel luminance but others such as contrast or hue or other features using other color spaces could be used. For our purposes we will not be using contrast but since our image is engineered for this task we can skip the contrast extraction that needs to be in place for real world images and poduce images that has clear distinct lines present in the raw pixel data (since the dimensions are small a line in the image will typically only be a few pixels wide so there would be no difference if we took the contrast of pixels or the luminance direclty).. If the image is black and white it is straight forward to extract the luminance as most often the pixel luminace is stored directly. Once we have ound the range say minimum is 0 and max is 255 we can construct an event queue where the pixel intensity determines its place in the queue and its assoiated delay. The conversion mapping can be linar where each pixel gets its luminace multiplied by a scalar other usefull mappings can be logarithmic or exponential.

```
procedure intensity_to_delay_encoding(image, T_max=100, T_min=0):
    normalized_image = normalize(image)
    spike_times = T_max - (T_max - T_min) * normalized_image
    return spike_times
```

Algorithm 1: Linear intensity to dely encoding

```
procedure intensity_to_delay_encoding(image, T_max=100, T_min=0):
    normalized_image = normalize(image)
    spike_times = T_max - (T_max - T_min) * normalized_image
    return spike_times
```

Algorithm 2: Logarithmic intensity to dely encoding

In deeper layers the meaning of the information may be obscured as is common with deep neural networks, however temporal encoding should if the neuron model is designed with this sceme in mind should produce shorter delays for greater values if a deeper layer neuron reacts to a box and its neigbour in the same layer reacts to a circle and the image is a rounded box (but more box than circle) then the neuron responind to a box should emit a spike faster than the neuron responding to a circle.

3.2.2 • DECODING

Decoding the neural code requres an adequate neuron model that uses the neural code to run useful computations. As discussed in Section 2.2 a neuron should have the following charecteristcs:

- Acuumulate spikes in some form (integrate)
- Fire when some treshold has been reached
- Leak the potential over time

We have seen that glif neuron is bio plausible and such a neuron is simple and can be realized easily on hardware the way. Using a temporal code the neuron has to differentiate between inputs in the time domain otherwise it cannot decode the information and cannot do useful work. Earlier inputs are encoded with larger values and should have a greater portion of summation going on inside the neuron. The neurons need to be desigend to work with the chosen encoding either at individual neuron level or population level

-If order matters (temporal code) the neuron must handle it Evidence for bio-plauibility can be found from eq 1 where the R(u) is dependent on the membrane potential

make the input exponentially weaker in proportion to the threshold. If the treshold is zero then the incoming spike does not get suppressed if the therhold is very close to max then the incoming spike gets more suppressed.

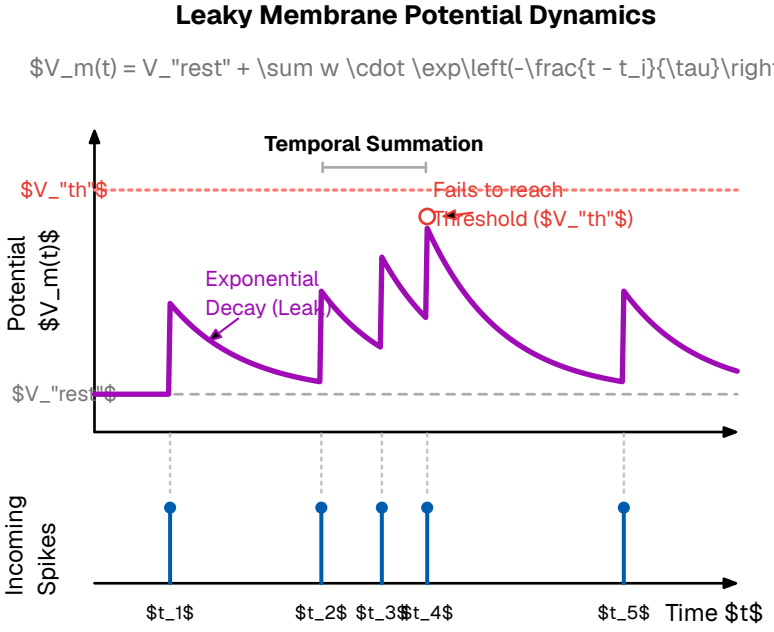


Figure 34: Neuron model where new incomming spikes have an exponential decaying influence on the potential

A counter starts when the first spike arrives

Needs a global reset or local like we talked about in the biological section about phase ambiguity

```
  integrate_and_fire(excitatory, inhibitory, threshold) → integer:
1   excitatory_spikes = []
2   if inhibitory is None:
3     inhibitory_spikes = []
4   else:
5     inhibitory_spikes = []
6   all_spikes = excitatory_spikes + inhibitory_spikes
7   if not all_spikes:
8     return None
9   all_spikes.sort(key=lambda x: x[0])
10  integrated_potential = 0.0
11  firing_time = None
12  for time, spike_type in all_spikes:
13    integrated_potential += spike_type
14    integrated_potential = max(0, integrated_potential)
15    if integrated_potential ⩾ threshold:
16      firing_time = time
17      break
18  return firing_time
```

Algorithm 3: Logarithmic intensity to dely encoding

In a time to first spike scheme of we care about the order (the relative values since information is stored in time and order) we have to use weights and a neuron model that distinguish between inputs arriving earlier than others. I present a scheme where the first neuron that arrives starts a linear count where the slope of the counter is the weight additional inputs will increase or decrease the slope according to their weight. We can see that neurons arriving earlier will get more time to increase the counter and thus will carry a higher value. If the counter reaches a threshold the neuron will fire. The astute will notice that in this scheme the neuron will fire even for the smallest stimulus since the counter will count up a non zero value and eventually reach the threshold, to mitigate this we can simply say that if the counter is too slow the neuron will not fire we will see later that this scheme satisfies the criteria above.

The problem with this decoding is for strong stimuli we would ideally make the neuron respond immediately and fire, but it has to wait until the counter has reached the threshold to fix this we can also add the weight of the input directly to the potential while also starting a counter. Now if early strong inputs arrive they will fill up the potential and make the neuron fire almost immediately. Small inputs wil take some time

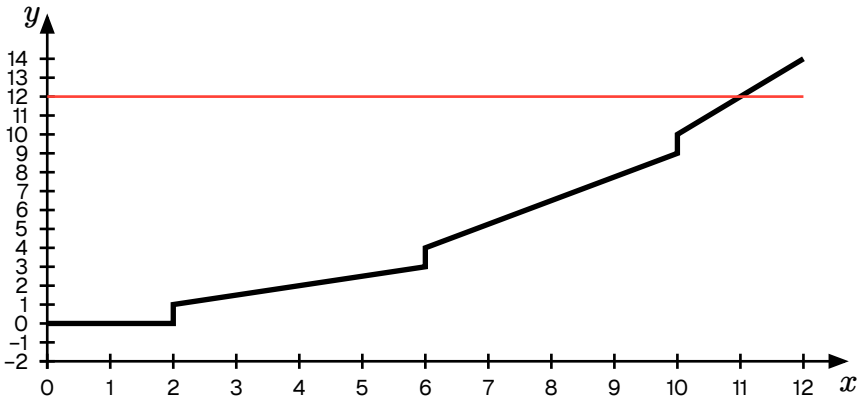


Figure 35: Proposed simplifed layout of a SNN. The neurons are connected with hirearcical busses that allow for the network to be configured as a *small world network*

recall this equation. TTFS model should mathematically behave the same

$$I_{i(t)} = \sum_j w_{ij} \cdot S_{j(t)}$$

Leaky integrate and fire models seem the best bet, however complex dynamics like exponential decay and analog weights and potentials seem excessive, we might do without. Binary weights 1 for excitatory and and 0 for inhibitory. Stronger weights can be modeled with multiple parallel synapses

Another way which is also based on relative firing order of single spikes could be a passcode encoding. Such an encoding could work by having neurons only react to a sequence. It has an internal state machine of sorts and will only advance to the next state if recives the correct input in the correct order. This encoding does only care about relative order not relative timings.

3.3 • NETWORK ARCHITECHTURE

As the methods will be fitted to visual stimuli. Network topologies for these tasks are widely studdied and proved to be effective the topolygy we will be using is similar to CNN topology whiich is also inspired by the visual cortex in mamals. The idea is that early layers will pick up on very simple features like lines and curves that form eg around an object. The next layer might use the representation of lines and curves from the layer before to represent more complex shaped like boxes and circles. Furhter down the network more complex and abstract features can be represented this way. We will focus on fairly shallow networks with three layers to capture the simple shapes like boxes and circles present in the data. Both the CNN and the SNN will have the same architechture and number of parameters so that weights can be shared and the idea is that comparisons should be more fair.

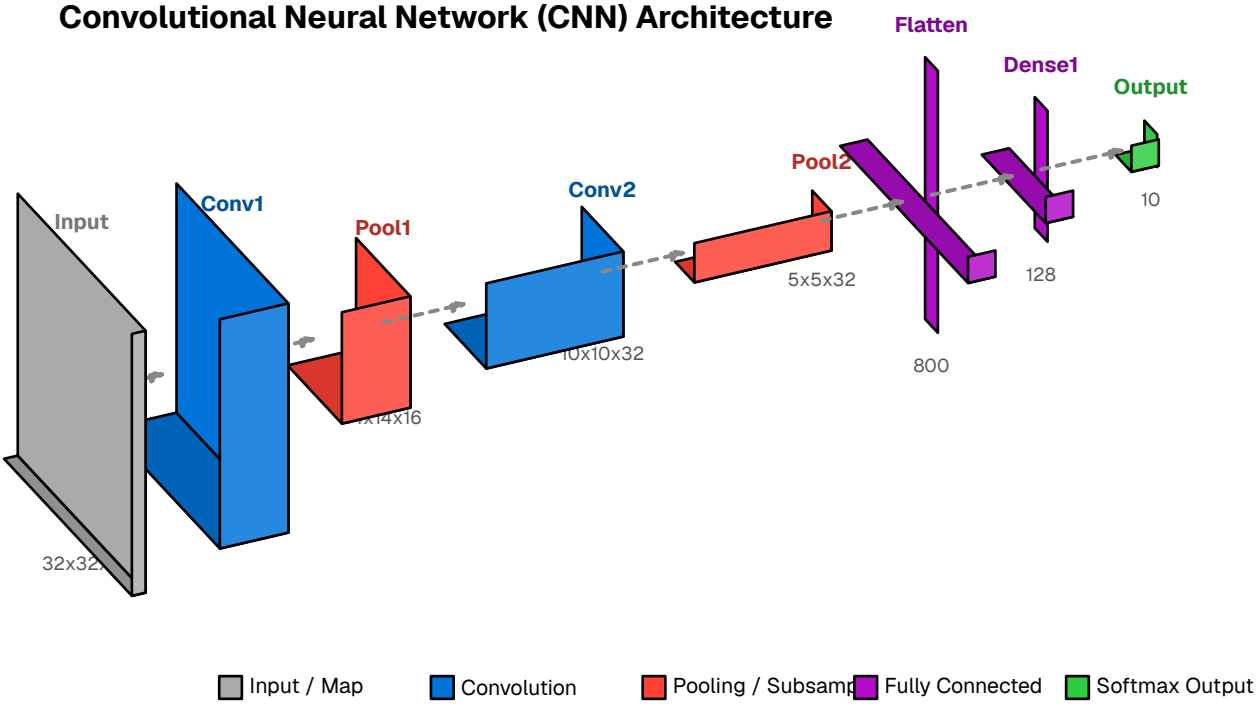


Figure 36: Network arhcitechture for the task at hand

Using the same architechtire for CNN and SNN and sharing weights is not mathe-mathically eqiuvalent but sharing weights is still usefull. The key distinction is that the SNN uses lateral inhibition the cnn does nececarly do, altough max pooling works in the same way.

The input layers match the input dimension

use 3x3 filters since features are not that big

use 3 channels, each channel is used for a spesicic local feature, use wta/max pooling to choose the most prominent feature in a local region (lateral inhibition. A local feature like vertical diagonal or horisontal line is “chosen” in that region)

3.4 • PROPOSED LEARNING ALGORITHMS

start with fully connected and then run strengthen connections that have “statistical significance” (not random / patterns) or is correlated to something (reward signal) at the end of an epoch prune connectionn making the network more effecient. This is similar to how the brain does it when sleeping

Do some math with references to the optimasation section

If the post neuron fires then we should strengthen the weights. Synapses gets grown at random this might be inneficient but it is highly parallelizable an idea could be to grow synapses with a kind of gravity where post synapses has a pulling effect if they are already strong

The encoding fit for images are a kind of population code where a pattern comes at the same time. The relative timing between patterns is used for strength earlier patterns are stronger. In a inhibitory network this allows for winner take all

learning with binary weights

learning with more steps of quantized or continous weights

Say we want to detect the pattern ABC and the pattern ABD. First of all if the order does not matter set all the weights equal. If the order does matter the weights determine the order. Now if a neuron learns pattern ABC so well that it learns to fire on only AB then it can fire faster. However if a second neuron wants to learn ABD then inhibition from the AB neuron prohibits it. A solution can be that if a neuron originally learned ABC but now fires on AB but stil has a strong weight on C it should remember this and if it fires on AB but then C does not arrive it should be like “oh, C did not show maybe I am wrong to fire early” eg. Decrease weights for A and B It predicts!

A second way is to have a hierarchy with bypass. So one layer detects only AB then the next layer has bypass of the first layer and the second combining AB and C or D

A second problem is how to decode order. When do we start the decreasing timer, how fast, should it be in time or in amount of spikes, what to do with phase? The phase should correct itself. The weights need to be as presise as the timing of the spikes? Or we could make the neuron sensitivity proportional to its inverse potential and add leaking

```
start with a collection of neurons with arbitrary connections
if a pre-synaptic neuron fires then
    it has a chance to grow a synapse to a random post-synaptic neuron
if a post-synaptic neuron fires then
    strengthen all connections to pre-synaptic neruons that fired before
    remove connections to pre-synaptic neurons that did not fire or fired after
    ⓘ a neuron can be both pre-synaptic and post-synaptic
```

Algorithm 4: Unsupervised local learning rule for induvidual neurons. Based on STDP

```
probability of growing a synapse is inversely
proportional to the amount it already has
earlier firings should get a better chance to grow synapses,
although this is regulated by inhibitory action
```

Algorithm 5: Growing rules for synapses

3.5 • SIMULATION

We have discussed the benefits of co algorithm design and designing new specialed computer hardware that run neuromorphic algorithms directly on the substrate via gates or analaog elements and exotic new materials. However developing biologically inspired computers and algorithms on cheap and avaliable CPU and GPU hardware is a great way to quicly iterate and test

Altough many simulation and software packages exists as outlines in Section 2.4.6. The methods in this thesis has been tested using custom simulaton software for full control

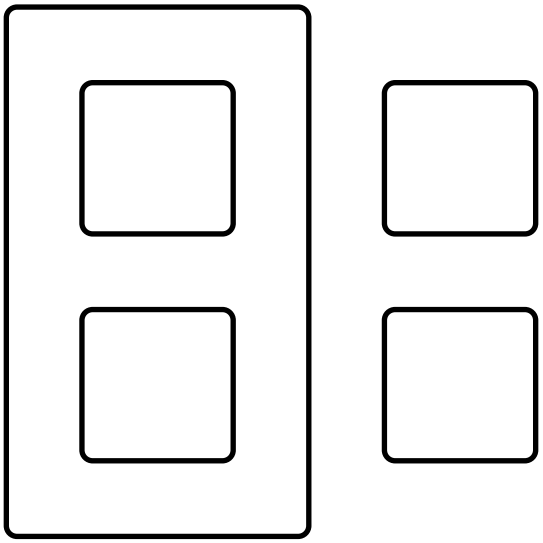


Figure 37: Simulator architechture block diagram

The simulator runs an event loop Spikes are pushed to a heap The simulator can run both on CPU and GPU, When running on CPU the spikes are pushed to a heap, when running on GPU spikes are pushed to a adress event bus the algorithms presented in this thesis are highly paralizable It is built from the ground up using the Vulkan API

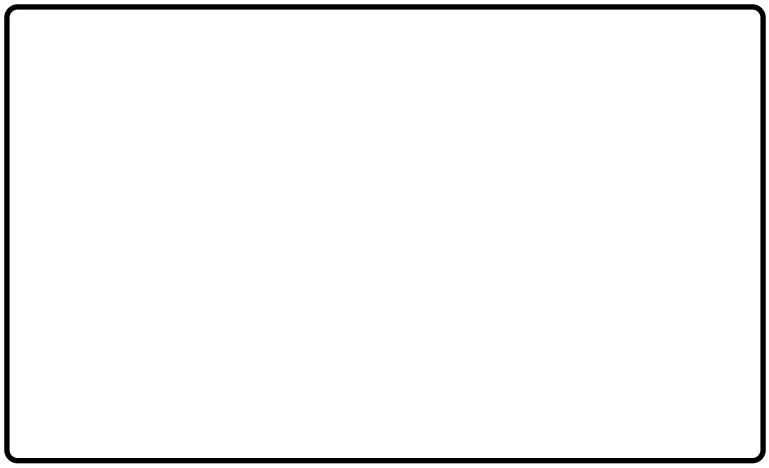


Figure 38: In-memory

3.6 • METRICS

To test and verify the methods we need a way to measure the performance for classification tasks accuracy recall and . is often used. This works great for supervised learning. For unsupervised learing ...

Accuracy and recall measures effectivness but as the goal of this thesis is to improve efficiency we need a way to mesaure the resource usage. There are direct ways to do this like measure power draw and data usage however we do not have the resources to set up such a test rig. Another way is the measure and theroize about number of operations as an indirect measure of resoource usage. This has several accuracy issues but can give an estimate. The first issue is that this is not an apples to apples comparison

4 • RESULTS

4.1 • INFERENCE

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

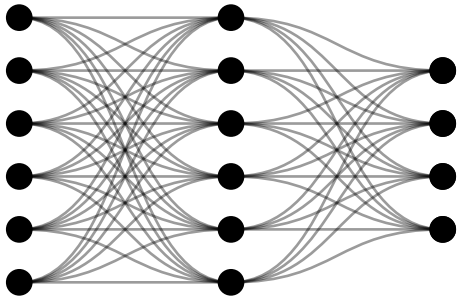


Figure 39: Neural network before learing

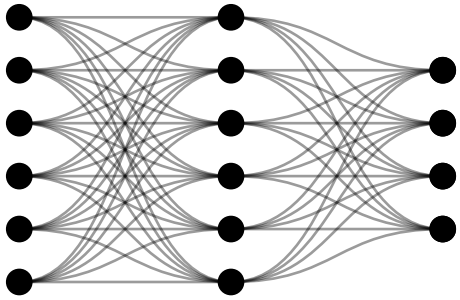


Figure 40: Neural network during learning

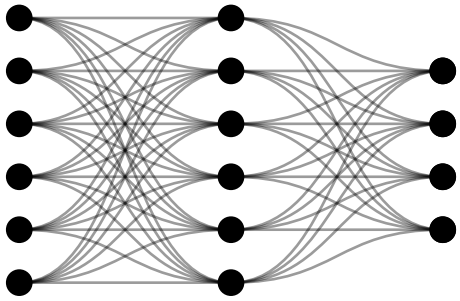


Figure 41: Neural network after learning

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest,

si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

4.2 • TRAINING

0	0	1	1
1	1	1	1

Table 1: Number of operations

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum

impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest,

si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

5 • DISCUSSION

In this section we discuss the

In the engineered test enviroment we used pixel intensity directly and for this toy example it works quite well since the feeatures we are interesting in are eesy to read in a low resoluton simple enviroment. However for real world vision tasks pixel intensity is not the best features to feed into the algorithm, it is better to use local contrast since that is what determines object outlines and other interesting elements of an image. It is possible to use an SNN to compute local contrast from pixel intsity, in fact that was part of the development of the algorithm. For a TTFS aproach it gets very difficult to do this effeciently since to get the normalized contrast (true contrast indepenendt of luminance) you have to wait for all signals to arrive even the complete absense of light completly wasting the potential of TTFS. To fix this a dedicated chip that outputs TTFS contrast directly can be used. A normal difference of lumince gives you a absolute measure of contrast where differences of brighter pixels gives a larger contrast where differences between two darker pixels gives a lower contrast. Contrast should be relative (a ratio) so by converting luminace to log-space first and then taking difference will give a true contrast independent of luminance intensity

We see a trade-off between the ability to learn and speed of learing and forgetting. Synaptic plasticitiy must be tuned in order for the right learing enviroment to form

representing contrast is difficult with a ttfs scheme, better to use special sensors and pass the encoded contrast in ttfs to a neuromophic processor

representing position in a ttfs scheme proved difficult, presise timings and clever encodings are needed to map a delay to a coordinate. in biology section we brefly discussed population coding in the visual and motor cortex where neurons are orthogonal and encode a direction, however these schemes seem to work best with rate encoding as the intensity of a direction is more straight forward to encode, simply increase firing rate for that neuron, with ttfs we need a reference signal and have to wait for the slowest neuron. for numerical values rate encoding seems best for categorical and decicion making ttfs is good.

Another way may be to have a hierarcy of “space cells” say each in a grid of 8x8 followed by another 8x8 that way we have 8x8 + 8x8 rather than 32x32 ofc we still have limited resoulution at the borders

5.0.1 • PERCEPTRON EQUIVALENCE

The perceptron equation can be obtained with a ttfs using the inverse of firing times

$$T = \sum \frac{w}{t}$$

Equation 9: Weigthed sum

In a time to first spike scheme of we care about the order (the relative values since information is stored in time and order) we have to use weights and a neuron model that distinguish between inputs arriving earlier than others. I present a scheme where the first neuron that arrives starts a linear count where the slope of the counter is the weight additional inputs will increase or decrease the slope according to their weight. We can see that neurons arriving earlier will get more time to increase the counter and thus will carry a higher value. If the counter reaches a threshold the neuron will fire. The astute will notice that in this scheme the neuron will fire even for the smallest stimulus since the counter will count up a non zero value and eventually reach the threshold, to mitigate this we can simply say that if the counter is too slow the neuron will not fire we will see later that this scheme satisfies the criteria above.

The problem with this decoding is for strong stimuli we would ideally make the neuron respond immediately and fire, but it has to wait until the counter has reached the threshold to fix this we can also add the weight of the input directly to the potential while also starting a counter. Now if early strong inputs arrive they will fill up the potential and make the neuron fire almost immediately. Small inputs wil take some time

recall this equation. TTFS model should mathematically behave the same

$$I_{i(t)} = \sum_j w_{ij} \cdot S_{j(t)}$$

Leaky integrate and fire models seem the best bet, however complex dynamics like exponential decay and analog weights and potentials seem excessive, we might do without. Binary weights 1 for excitatory and and 0 for inhibitory. Stronger weights can be modeled with multiple parallel synapses

Another way which is also based on relative firing order of single spikes could be a passcode encoding. Such an encoding could work by having neurons only react to a sequence. It has an internal state machine of sorts and will only advance to the next state if recives the correct input in the correct order. This encoding does only care about relative order not relative timings.

easy with rate code

need global information to normalize

using ttf is difficult

using negative image - still problem with absolute contrast

can be done locally with speial sensors and using logarithmic intensity to delay

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea

variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

5.1 • FUTURE WORK

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

6 • CONCLUSION

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque

doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

BIBLIOGRAPHY

[1] ?, “Placeholder,” 2025.